



Documaker

Documaker Server System Reference

version 11.3

Skywire Software, L.L.C.
3000 Internet Boulevard
Suite 200
Frisco, Texas 75034
www.skywiresoftware.com

Phone: (U. S.) 972.377.1110
(EMEA) +44 (0) 1372 366 200
FAX: (U. S.) 972.377.1109
(EMEA) +44 (0) 1372 366 201
Support: (U. S.) 866.4SKYWIRE
(EMEA) +44 (0) 1372 366 222

support@skywiresoftware.com

PUBLICATION COPYRIGHT NOTICE

Copyright © 2008 Skywire Software, L.L.C. All rights reserved.

Printed in the United States of America.

This publication contains proprietary information which is the property of Skywire Software or its subsidiaries. This publication may also be protected under the copyright and trade secret laws of other countries.

TRADEMARKS

Skywire® is a registered trademark of Skywire Software, L.L.C.

Docucorp®, its products (Docucreate™, Documaker™, Docupresentment™, Docusave®, Documanage™, Poweroffice®, Docutoolbox™, and Transall™), and its logo are trademarks or registered trademarks of Skywire Software or its subsidiaries.

The Docucorp product modules (Commcommander™, Docuflex®, Documerge®, Docugraph™, Docusolve®, Docuword™, Dynacomp®, DWSD™, DBL™, Freeform®, Grafxc commander™, Imagecreate™, I.R.I.S.™, MARS/NT™, Powermapping™, Printcommander®, Rulecommander™, Shuttle™, VLAM®, Virtual Library Access Method™, Template Technology™, and X/HP™) are trademarks of Skywire Software or its subsidiaries.

Skywire Software (or its subsidiaries) and Mynd Corporation are joint owners of the DAP™ and Document Automation Platform™ product trademarks.

Docuflex is based in part on the work of Jean-loup Gailly and Mark Adler.

Docuflex is based in part on the work of Sam Leffler and Silicon Graphic, Inc.

Copyright © 1988-1997 Sam Leffler.

Copyright © 1991-1997 Silicon Graphics, Inc.

Docuflex is based in part on the work of the Independent JPEG Group.

The Graphic Interchange Format® is the Copyright property of CompuServe Incorporated. GIF® is a Service Mark property of CompuServe Incorporated.

Docuflex is based in part on the work of Graphics Server Technologies, L.P.

Copyright © 1988-2002 Graphics Server Technologies, L.P.

All other trademarks, registered trademarks, and service marks mentioned within this publication or its associated software are property of their respective owners.

SOFTWARE COPYRIGHT NOTICE AND COPY LIMITATIONS

Your license agreement with Skywire Software or its subsidiaries, authorizes the number of copies that can be made, if any, and the computer systems on which the software may be used. Any duplication or use of any Skywire Software (or its subsidiaries) software in whole or in part, other than as authorized in the license agreement, must be authorized in writing by an officer of Skywire Software or its subsidiaries.

PUBLICATION COPY LIMITATIONS

Licensed users of the Skywire Software (or its subsidiaries) software described in this publication are authorized to make additional hard copies of this publication, for internal use only, as long as the total number of copies does not exceed the total number of seats or licenses of the software purchased, and the licensee or customer complies with the terms and conditions of the License Agreement in effect for the software. Otherwise, no part of this publication may be copied, distributed, transmitted, transcribed, stored in a retrieval system, or translated into any human or computer language, in any form or by any means, electronic, mechanical, manual, or otherwise, without permission in writing by an officer of Skywire Software or its subsidiaries.

DISCLAIMER

The contents of this publication and the computer software it represents are subject to change without notice. Publication of this manual is not a commitment by Skywire Software or its subsidiaries to provide the features described. Neither Skywire Software nor its subsidiaries assume responsibility or liability for errors that may appear herein. Skywire Software and its subsidiaries reserve the right to revise this publication and to make changes in it from time to time without obligation of Skywire Software or its subsidiaries to notify any person or organization of such revision or changes.

The screens and other illustrations in this publication are meant to be representative, not exact duplicates, of those that appear on your monitor or printer.

Contents

Chapter 1, Introduction

- 2 System Overview
- 3 Rules Publishing Solution Overview
- 4 Document Automation Evolution
 - 4 Stage 1 - paper automation
 - 5 Stage 2 - workflow automation
 - 6 Stage 3 - paperless information automation
- 6 Document Automation Goals
- 8 System Benefits

Chapter 2, Understanding the System

- 10 Processing Overview
- 13 Processing Options
- 14 Using Banner Processing
 - 14 Enabling banner processing
 - 15 Specifying banner forms and scripts
 - 16 Processing logic
 - 17 DAL functions
 - 17 Banner processing example
- 19 Multi-step Processing
 - 19 Creating Transaction Records
 - 20 File Summary
 - 21 Processing Transactions
 - 22 Output Files for GenPrint
 - 22 Output Files for GenWIP
 - 22 Output Files for GenArc
 - 23 File Summary
 - 25 Creating Print Spool Files
 - 26 File Summary
 - 27 Sending Incomplete Transactions to WIP
 - 28 File Summary

29 Archiving Transactions	
30 File Summary	
30 Rules Used in Multi-Step Processing	
31 Restarting the GenData Program	
31 RULCheckTransaction rule	
32 RestartJob rule	
32 INI options	
33 Controlling GenTrn Processing	
35 Single-step Processing	
35 Creating and Processing Transaction Records	
36 System Settings and Resources	
37 Creating Print Files	
38 File Summary	
39 MultiFilePrint callback function	
40 Mapping Fields with XPath	
41 Running Archive in Single-Step Processing	
41 Running WIP in Single-step Processing	
41 Rules Used in Single-step Processing	
42 Archive	
42 BatchByRecipINI	
42 BatchByPageCount	
42 BuildMasterFormList	
42 ConvertWIP	
43 InitArchive	
43 InitConvertWIP	
43 InitPrint	
43 InitSetRecipCache	
43 NoGenTrnTransactionProc	
43 PageBatchStage1InitTerm	
43 PaginateAndPropagate	
43 PrintFormset	
44 ProcessQueue	
44 StandardFieldProc	
44 StandardImageProc	
44 WriteNAFile	
44 WriteOutput	
44 WriteRCBWithPageCount	
45 Single-step Processing Example	
45 Base rules	
46 Base form set rules	

46	Base image rules
46	Base field rules
47	Using IDS to Run Documaker
48	Writing Unique Data into Recipient Batch Records
49	Optional formatting information
50	Example
52	BANNER.DAL
55	Using Class Recipients
57	Running Documaker Using XML Job Tickets
58	Handling 2-up Printing
58	2-up printing with single-page forms
59	2-up printing with multi-page forms
59	Changing the INI File
60	Creating the TWOUP control group
61	Creating the Added_Fonts control group
61	Changing the Recipient Batch DFD File
62	Rules Used for 2-up Printing
62	AddLine
62	AddTextLabel
62	ForceNoImages
62	GetRCBRec
62	InitMerge
62	InitPageBatchedJob
63	MergeAFP
63	ParseCommentExample
63	PrintData
63	ProcessRecord
64	Placing the 2-up Rules in the JDT File
65	2-up Processing Example
65	2upbycnt.bat
65	2upstep1.ini
65	2upstep2.ini
65	2upstep3.ini
66	Running the GenData Program
66	Step 1 - Using the AFGJOB1.JDT file
67	Step 2 - Using the AFGJOB2.JDT file
68	Step 3 - Using the AFGJOB3.JDT file
69	Splitting Recipient Batch Print Streams
69	Splitting batches by sheet count

70	Creating PDF output
70	DAL functions
71	DeviceName
71	SetDeviceName
71	BreakBatch
72	UniqueString
72	Using DAL to Manipulate File Names
73	FileDrive
73	FilePath
73	FileName
73	FileExt
74	FullFileName
75	Controlling WIP Field Assignments
78	Multi-mail Processing
78	Changing the RCPDFDFL.DAT and TRNDFDFL.DAT Files
79	Setting Up the FSISYS.INI File for Multi-mail Processing
81	Adding and Removing Pages
81	Using Custom Code
81	Adding pages
81	Removing pages
81	Using DAL Scripts
81	Adding pages
82	Removing pages
83	Using IDS
83	Adding pages
83	Removing pages
84	Adding Indexes and Tables of Contents
85	Run-Time Options
85	GenData Command Line Options
85	GenPrint Command Line Options
86	GenTrn Command Line Options
86	Debugging Options
87	Noting font IDs of zero
88	Grouping Print Batches
89	Using INI Built-In Functions
89	~GetEnv
90	~Platform
90	~OS

91	~DALRUN
	~DALVAR
91	~Encrypted
92	~ProcessID
93	~WIPField
93	Accessing WIP Fields
	94 Formatting arguments
	96 Specifying locales
	97 Using the ~Field function
98	Using XML Files
	98 Handling Overflow
	99 Triggering Forms and Images
100	Using XPath
	100 XPath Syntax
	100 Axes
	101 Symbols
	101 Functions
	102 Expressions
	103 Using the XPath Testing Utility
	103 Example 1
	104 Example 2
	104 Example 3
	104 Example 4
	104 Example 5
	105 Example 6
	105 Example 7
	105 Example 8
	106 Example 9
	106 Example 10
	107 Example XML File
108	

Chapter 3, Implementing Your System

110	Why Use a Methodology?
	111 Phase 1 - Define the requirements
	111 Phase 2 - Create the detail forms requirements
	111 Phase 3 - Build the Master Resource Library
	111 Phase 4 - Install and configure the system

- 111 Phase 5 - Test the system
- 111 Phase 6 - Go live
- 112 Gathering Information
 - 112 Understanding Your Niche
 - 112 Understanding Your Organization
- 113 Roles and Responsibilities

Chapter 4, Setting Recipients and Copy Counts

- 116 Concepts
- 117 Key Files
 - 117 Transaction Trigger Table
 - 117 Trigger Levels
 - 117 Form Set Definition Table
- 118 Trigger Table Record Format
- 120 Specifying the Transaction Trigger Table
- 121 How Transaction Triggering Works
 - 122 Image Level Triggers
- 124 Form Level Triggers
- 126 Master and Subordinate Images
 - 126 Marking Subordinate Images
 - 127 Marking Master Forms
- 128 Examples
 - 129 Specifying Copy Counts and Images
 - 130 FORM.DAT file
 - 130 SETRCPTB.DAT file
 - 130 POL file
 - 131 Using Transaction Codes
 - 131 FORM.DAT file
 - 131 SETRCPTB.DAT file
 - 131 POL file
 - 132 Setting Up Search Mask and Images
 - 132 FORM.DAT file
 - 132 SETRCPTB.DAT file
 - 133 POL File
 - 134 Using the RECIPIF Rule

134	FORM.DAT file
134	SETRCPTB.DAT file
135	POL file
136	Using Automatic Overflow
136	FORM.DAT file
136	SETRCPTB.DAT file
136	POL file
138	Using Forced Overflow
138	FORM.DAT file
138	SETRCPTB.DAT file
138	POL file
139	Setting Search Masks and Recipients
139	FORM.DAT file
139	SETRCPTB.DAT file
139	POL file
140	Using the Set Recipient Table and Extract Files
141	Formatting Search Masks
141	Spaces
141	Commas
141	Tildes
142	Parentheses
142	Using the OR condition
142	Using the NOT condition
142	Using AND and OR conditions
143	Sorting Forms by Recipient
144	INI files
144	Sort tables
145	Summary

Chapter 5, Working with Fonts

148	Understanding Font Concepts
148	Font Terminology
149	National language terminology
151	How Characters are Represented
151	Bitmap Fonts
151	Scalable Fonts
152	TrueType
152	PostScript

152	How Computers and Printers Use Fonts
154	Using Code Pages
155	ASCII Code Pages
157	EBCDIC Code Pages
158	Character Sets
158	Determining Characters Used in a Printer Font
160	Code Page Names
163	Types of Fonts
163	Using Screen Fonts
163	Font Substitution in Windows
164	Installing Screen Fonts in Windows
164	Using Printer Fonts
164	AFP
164	Coded fonts
164	Code pages
164	Character sets
164	Metacode
165	PCL
165	PostScript Fonts
165	TrueType Fonts
165	Adding Printer Fonts to a Font Cross-reference File
166	Using System Fonts
167	Font Cross-reference Files for Monotype Fonts
167	HPINTL.FXR, HPINTLSM.FXR
167	REL95.FXR, REL95SM.FXR
167	REL102.FXR, REL102SM.FXR
167	REL103.FXR, REL103SM.FXR
169	Using Custom Fonts
170	On AFP printers
171	On Xerox Metacode printers
171	On PCL printers
171	On PostScript printers
172	Using Font Cross-Reference Files
173	How FXR Settings Affect Display and Print Quality
174	Maintaining FXR Files
174	Choosing a Font Cross-reference File
176	International Language Support
176	Using the ANSI Code Page for PC Platforms

177	Using Code Page 37 for EBCDIC Platforms
178	Using International Characters
179	Converting Text Files from one Code Page to Another
180	Setting Up PostScript Fonts
183	Fonts for PDF Files
184	Font Naming Conventions
185	Using Font Manager
185	Starting Font Manager
185	From Image Editor
185	From Docucreate
186	Working with the Font List
186	Selecting Fonts
187	Deselecting Fonts
187	Filtering the List of Fonts
188	Adding Fonts to a Font Set
190	Description tab
193	Dimensions tab
195	Printers tab
199	Other tab
199	Copying Font Information
200	Editing Font Information
201	Converting Fonts
203	Converting fonts from other vendors
203	Working with multiple printers
206	Deleting Fonts
206	Inserting Fonts
208	Inserting bitmap fonts and FXR files
209	Inserting PostScript and TrueType Fonts
212	Choosing Screen Fonts
214	Generating Files using Font Manager
214	Generating an FNT File
215	Generating an XRF File
215	Generating PFM Files from an FXR file
218	Mapping Fonts for File Conversions

Chapter 6, Setting Up Printers

220 AFP Printers

220 AFP INI Options

224 Printing highlight colors

225 AFP Printer Resources

225 FormDef

225 Fonts

225 Monotype fonts

225 Overlays

225 Page segments

225 AFP 2-up support

226 AFP Troubleshooting

226 Floating image limitations

226 Objects extending beyond the edges

226 Conflicts between page and form orientation

227 Multi-page FAP limitation

227 Printing rotated variable fields

227 AFP 240 dpi print problems

229 Metacode Printers

229 Required JSL INI Options

230 JDLName

230 JDEName

231 DJDEIden, DJDEOffset, and DJDESkip

231 JDLCode

231 JDLData

231 JDLHost

232 Additional Required INI Options

232 OutMode

232 ImageOpt

232 CompressMode

233 CompileInStream

233 Device

234 RelativeScan

234 Specifying Installable Functions

234 Optional INI Options

234 Setting the end of the report

234 Starting new pages

235 Jogging pages

236 Specifying spot color

236 Chart performance and print quality

236 Optimizing Metacode print streams

237 Using a common font list

238	Setting a default paper size
239	Inline graphic logo performance and print quality
239	Adding color to charts
239	Using named paper trays
239	Specifying the printer model
239	Specifying the resolution
239	Displaying console messages
240	Stapling forms
240	Duplex switching
241	Using VSAM to store resources
241	PrintViewOnly
241	Caching files to improve performance
242	Using the loader
243	Using the Class option
243	Adding user-defined DJDE statements
244	Using third-party software to read Metacode files
245	Metacode Printer Resources
245	Fonts
245	Forms
245	Images
245	Logos
245	Metacode Limitations
245	Xerox images
245	HMI support
245	Changing the paper size on the 4235 printer
245	Xerox forms
246	Metacode Troubleshooting
246	Unexpected color output
246	Unexpected black and white output
247	Highlight color should match the PrinterInk option
247	Logo orientation
247	Output catching up with the input
247	Printing rotated variables
248	Multi-page images
249	Operator command, FEED, causes duplex problems
249	Line density errors
250	Output data length validation
250	Using Xerox Forms (FRMs)
251	BARRWRAP
252	Transferring Files from Xerox Format Floppies
253	PCL Printers

253 PCL INI Options	
256 Using PCL 6	
257 Printing Under Windows	
257 Using High-Capacity Trays 3 and 4 on HP 5SI Printers	
259 Using a staple attachment	
259 Using Simple Color Mode	
259 Marking objects to print in color	
260 Specifying the highlight color to use	
260 Printing on different types of printers	
260 Creating Compressed PCL Files	
261 Adding Printer Job Level Comments	
261 Adding Data for Imaging Systems	
262 PCL Printer Resources	
262 Fonts	
262 Overlays	
263 PostScript Printers	
263 PostScript INI Options	
266 Printing under Windows	
266 Generating PostScript Files on OS/390	
266 Adding DSC Comments	
267 Stapling Forms	
269 PostScript Printer Resources	
269 Fonts	
269 Overlays	
269 PostScript Printer Definition (PPD) Files	
270 Using the GDI Print Driver	
270 How it works	
271 The GDI driver and GenPrint	
272 GDI Printer Driver INI Options	
274 Avoiding Problems with FAX Drivers	
275 Using Pass-through Printing	
277 Creating PDF Files	
278 Creating RTF Files	
278 Generating separate files	
279 Adding or removing frames	
279 Creating form fields	
280 Setting margins	
280 Removing the contents of headers and footers	
281 Emailing a Print File	

281	Creating EPTLIB print files for Documaker Workstation
282	Creating EPTLIB print files for Documaker Server
284	Creating PDF print files
284	Overriding attached files
284	Using email aliases
285	Choosing the Paper Size
286	US Standard Sizes
287	ISO Sizes
287	ISO A sizes
288	ISO B sizes
289	ISO C sizes
290	Japanese Standard Sizes
291	Printer Support for Paper Sizes
295	Paper Sizes for AFP Printers
297	Creating Print Streams for Docusave
297	Archiving AFP Print Streams
298	Archiving Metacode Print Streams
299	Archiving PCL Print Streams
299	Using DAL Functions
301	Adding TLE Records
302	Handling Multiple Paper Trays
302	For PCL printers
302	For PostScript printers
303	For GDI printers
304	For AFP printers
304	For Metacode printers
304	Including Tray Selections in a Print Stream Batch

Chapter 7, Setting Up Error Messages and Log Files

306	Overview
307	Configuring the Message System
307	Enabling and Disabling Messages
308	Logging INI Files and Options Used
308	Clearing Messages
308	Defining the Output Message Files
309	Initializing the Output Message Files

- 310 Turning Off Date Stamps
- 310 Controlling the Translation Process
- 311 DBLib Trace Messages
- 312 Creating Messages
 - 312 Using the RPErrProc and RPLogProc Functions
 - 312 RP Struct
 - 312 Message Types
 - 313 Message Number
 - 313 Assigning numbers to custom messages
 - 314 Using Message Tokens
 - 315 Setting Up Message Text
 - 316 Message examples
 - 316 Undefined tokens
 - 316 Adding a new line
 - 317 Determining where the message originated
- 318 Using the Message Token File

Chapter 8, Archiving and Retrieving Information

- 322 Terminology
 - 322 Files and tables
 - 322 Commit
 - 322 Rollback
 - 322 GenArc
 - 322 AFEMAIN
 - 322 CARFILE
 - 322 APPIDX
 - 322 TEMPIDX
 - 323 CATALOG
 - 323 RESTART
 - 323 DFD
- 324 System Scenarios
 - 324 Scenarios for OS/390 (MVS)
 - 324 Scenarios for Windows 32-bit
 - 325 Scenarios for UNIX
- 326 Archive and Retrieval Features
- 327 Processing Overview
 - 327 DBASE IV

327 DB2	
327 SQL server	
327 Oracle	
327 Files GenArc Uses	
327 Input files	
327 Output files	
327 How the GenArc Program Works	
330 Running GenArc	
330 Logging archived transactions	
330 Archiving to a database	
331 Sorting records in a database	
331 Preparing SQL	
331 Command Line Options	
331 INI	
331 JOBID	
331 DPASSWD	
331 DUSERID	
332 OPASSWD	
332 OUSERID	
332 RESTART	
332 SQLID	
332 STOPREC	
333 Using the Restart Option	
335 Using GenArc with Documanage	
337 Forcing folder updates	
337 FSIUSER.INI sample	
339 APPIDX.DFD sample	
341 CARFILE.DFD sample	
342 Using the Oracle ODBC Driver	
342 CARFILE DFD	
344 Creating the Database and Tables	
347 Resolving Errors	
348 Viewing Archives in Documanage	
349 Using Multiple Simultaneous ODBC Connections	
351 Using WIP and the Archive Index File	
352 Formatting Archive Fields	
352 Converting the case of key fields	
352 Reformatting dates	
353 Storing a constant value	

- 354 Retrieving Archived Forms
 - 354 Files the Archive Module Uses
 - 354 Input files
 - 354 Output files
 - 354 Using the Archive Module
 - 355 Retrieval Options
- 357 Working with Documanager
 - 357 Using Documanager Data Type Support
 - 358 Setting Up Automatic Category Overrides

Appendix A, Setting Up Archive/Retrieval Configurations

- 360 DB2 Server on OS/390 —Windows Client
 - 360 Configuring the Server
 - 360 Getting the DB2 location name and LUNAME
 - 361 Defining the SNA server's APPC LU in VTAM
 - 361 Defining the DB2 Application Major Node in VTAM
 - 361 Setting Up the Windows 2000 Server (Middle Tier)
 - 361 Installing and configuring Microsoft's SNA Server
 - 363 Installing and Configuring Microsoft's SNA Server
 - 364 Configuring SNA Server 4.0 SP3
 - 366 Setting Up DB2 on a Windows 2000 Server
 - 366 Installing DB2 on a Windows 2000 Server
 - 366 Configure the DB2 instance
 - 366 Defining an OS/390 node
 - 367 Defining a system database entry
 - 367 Updating TCP/IP values on the Windows 2000 server
 - 367 Defining a database connection services entry
 - 367 Installing and Configuring DB2 on a Windows 2000 Server
 - 367 Defining an OS/390 system
 - 367 Defining a DB2 instance
 - 367 Defining an OS/390 database
 - 367 Setting Up Universal Database on Windows 2000
 - 367 Installing Universal Database
 - 368 Configuring Universal Database
 - 368 Updating TCP/IP-related Values on a Windows 2000 Server
 - 369 Common DB2 Errors
 - 369 Setting Up Clients
 - 369 Defining a DB2/2000 node

370	Defining a system database entry
370	Updating TCP/IP-related values on a Windows client
370	Setting Up the INI Options for the DB2 Driver
372	DB2 Server on Windows — Windows Client
372	Setting up a DB2 Database on the Server
373	Setting Up a Client for DB2 VERSION 6.1
373	Archiving to a remote DB2 database using an ODBC driver
373	Setting up an ODBC data source
374	Setting up INI options for the ODBC driver
375	Archiving to a Remote DB2 Database Using the Native DB2 Driver
375	Setting up a DB2 database
375	Setting up the INI options for the DB2 driver
377	DB2 Server and Client on Windows
377	Setting Up a DB2 Database
377	Setting up an ODBC data source
377	Setting up INI options for ODBC
379	Archiving to a Local DB2 Database Using the Native DB2 Driver
379	Setting up the DB2 database
379	Setting up the INI options for the DB2 driver
381	SQL Server on Windows — ODBC Client on Windows
381	Setting Up a Client
381	Setting up the INI options for ODBC
383	IDS on Windows —DB2 Archive on OS/390
385	Setting Up the DB2 Archive on OS/390
386	Creating an OS/390 Database
386	Updating TCP/IP Values on a Windows 2000 Server

Appendix B, System Files

390	Overview
392	Types of Files
392	BCH files
392	CAR files
392	DAT files
392	DBF files
392	DDT files

- 393 DFD files
- 393 Error files
- 393 Extract files
- 394 FAP files
- 394 Initialization files
- 394 JDT files
- 394 Log files
- 394 LOG files
- 394 MDX files
- 394 Transaction files
- 395 Resource Files
 - 395 FSISYS.INI file
 - 395 FSIUSER.INI file
 - 395 FAPCOMP.INI
 - 395 FORM.DAT file
 - 398 SETRCPTB.DAT file
 - 401 DFD files
 - 402 TRNDFDFL.DFD file
 - 402 RCBDFDFL.DFD file
 - 402 APPIDX.DFD
 - 403 .DDT files
 - 404 .JDT files
 - 404 Extract files
 - 406 DFD File Format
 - 406 Fields Group
 - 407 Field Description Group
- 409 Files Created by the GenTrn Program
 - 409 Transaction files
 - 409 Error files
 - 409 Log files
- 410 Files Created by the GenData Program
 - 410 NAFI.DAT file
 - 410 POLFILE.DAT file
 - 411 NEWTRN.DAT file
 - 411 Batch files
 - 411 MANUAL.BCH file
 - 411 Error batch
 - 411 Updated log, error, and message files
- 412 Files Created by the GenPrint Program
 - 412 Spool files

412 Updated log and error files

413 Files Created by the GenWIP Program

413 WIP.DBF file

413 WIP.MDX file

413 00000001.DAT file

413 00000001.POL file

413 UNIQUE.DBF file

414 Files Used by the GenArc Program

414 APPIDX.DBF file

414 APPIDX.DFD file

414 ARCHIVE.CAR file

414 APPIDX.MDX file

414 APPIDX.DFD file

Glossary

415 00000001.DAT File

415 00000001.POL File

416 AFP

416 ARCHIVE.CAR File

416 ARCHIVE.DBF File

416 ARCHIVE.DFD File

416 Base Product

416 .BCH Files

416 Batch Files

416 .CAR Files

417 Custom Solution

417 DAL

417 .DAT Files

417 .DBF Files

417 DDT Files

418 DESKJET.FXR File

418 .DFD Files

418 Distributed Resource Library

418 Duplex

418 ERRFILE.DAT

418 Error Batch

418 Error Files
419 External Database Editor
419 Extract Files
419 .FAP Files
419 FDB.DBF File
419 fetype
420 Field Database Editor
420 Fixed Data
420 Font Manager
420 Form
420 Form Set
420 Form Set Manager
421 FORM.DAT File
421 FSISYS.INI File
421 FSIUSER.INI File
421 .FXR Files
421 GenArc Program
422 GenData Program
422 GenPrint Program
422 GenTrn Program
422 GenWIP Program
422 Help Editor
423 Image
423 Image Editor
423 .INI Files
423 INTL.FXR
423 INTLSM.FXR
423 .JDT Files
423 Library Manager
424 Log Files
424 .LOG Files
424 Logo Manager
424 MANUAL.BCH File
424 Master Resource Library
424 Metacode
424 .MDX Files
424 NAFILE.DAT File

425 NEWTRN.DAT File
425 Objects
425 Overflow
425 Page
425 PCL
425 POLFILE.DAT File
425 PostScript
425 SETRCPTB.DAT File
426 Simplex
426 System Releases
426 System Patches
426 Table Editor
426 Transaction List
426 .TRN Files
426 TRNDFDFL.DFD File
426 UFSTSM.FXR File
426 UNIQUE.DBF File
427 Variable Data
427 WIP.DBF File
427 WIP.MDX
427 xBase

[429 Index](#)

Chapter 1

Introduction

Welcome to the Skywire Software's rules-based publishing solution. This Skywire Software product consists of a complete set of tools which provide solutions for all your form and document processing needs. The system includes these major components:

- Docucreate
- Documaker Workstation
- Documaker Server

This manual serves as a reference to Documaker Server. This chapter discusses the following topics:

- [System Overview on page 2](#)
- [Rules Publishing Solution Overview on page 3](#)
- [Document Automation Evolution on page 4](#)
- [System Benefits on page 8](#)

SYSTEM OVERVIEW

Documaker Server is part of the Skywire Software's Rules Publishing Solution, which also includes Docucreate, Documaker, and reusable resource libraries.

Documaker Server uses resources you create using Docucreate to process information and forms. This processing includes merging external data onto forms, processing data according to rules you set up, creating print-ready files, archiving data and forms, and, if applicable, sending incomplete forms to Documaker for completion by a user.

Forms can be completed using Documaker when user input is required or, if all of your information can be extracted from external data sources, you can set up Documaker Server to process forms without requiring user input.

Documaker Server can create print-ready files for a variety of printer languages including, AFP, PostScript, PCL, and Xerox Metacode printers. In addition, using Docupresentment, the system can produce output in Adobe Acrobat PDF format.

The following topic discusses the entire Skywire Software Rules Publishing Solution, its purpose, its underlying concepts and how it all works together to provide you with an enterprise-level solution to meet your document creation, processing, and storage needs.

RULES PUBLISHING SOLUTION OVERVIEW

Document automation is the basic concept underlying the system. An understanding of document automation helps you understand the purpose of the Skywire Software's Rules Publishing Solution.

Document automation replaces paper documents with electronic media. Generally, document automation is an integrated process within enterprise information systems.

The greatest challenge that document intensive industries face is the efficient processing of forms and documents. Moving toward the era of electronic information means finding workable solutions for the paper-to-electronic media replacement process. New business directions include developing ways to automate document handling processes, which extend beyond simply creating electronic output or print.

Document automation is rapidly becoming an integral part of today's business environment. The Rules Publishing Solution creates a total business solution which lets you automate both paper document processing and electronic document management.

Let's examine document automation outside the Rules Publishing Solution to build a knowledge base applicable to unique platforms. Then we can apply the basic concepts to the Rules Publishing Solution.

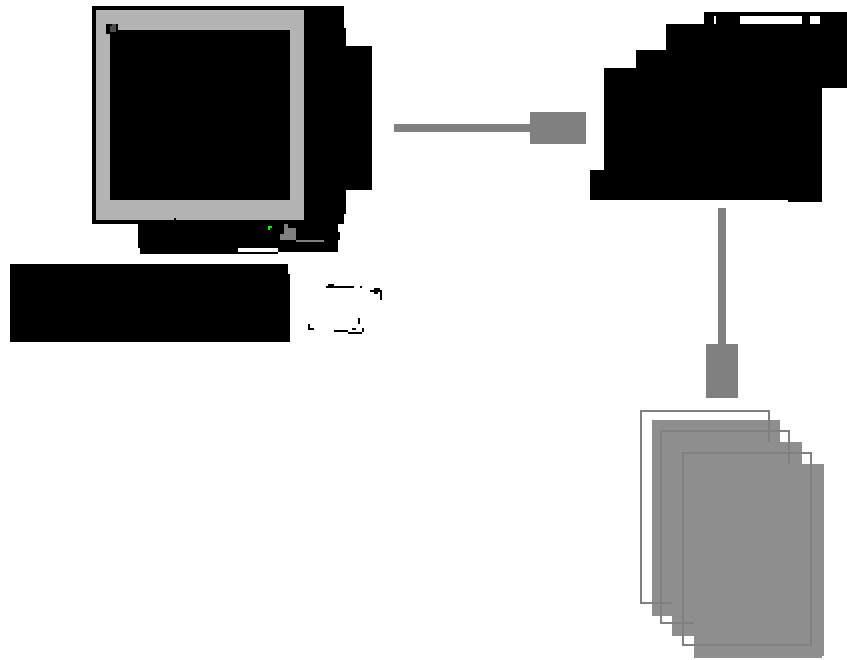
DOCUMENT AUTOMATION EVOLUTION

Through the years, document automation has moved in concert with technological evolution. The technological evolution has progressed from initial ideas and applications about forms processing, to the integrated management of electronic documents. The distinction between merely automating paper production and permanently integrating electronic processing and management is critical to understanding the technological evolution. This table shows the progression of document automation in the current environment.

Stage	Type of Automation	Components
1	Paper Automation	Business correspondence Forms processing Document assembly
2	Workflow Automation	Electronic mail Electronic data interchange Electronic funds transfer Integrated facsimile
3	Paperless Information Automation	Cooperative processing Enterprise indexing Integrated image processing Multimedia

Stage 1 - paper automation

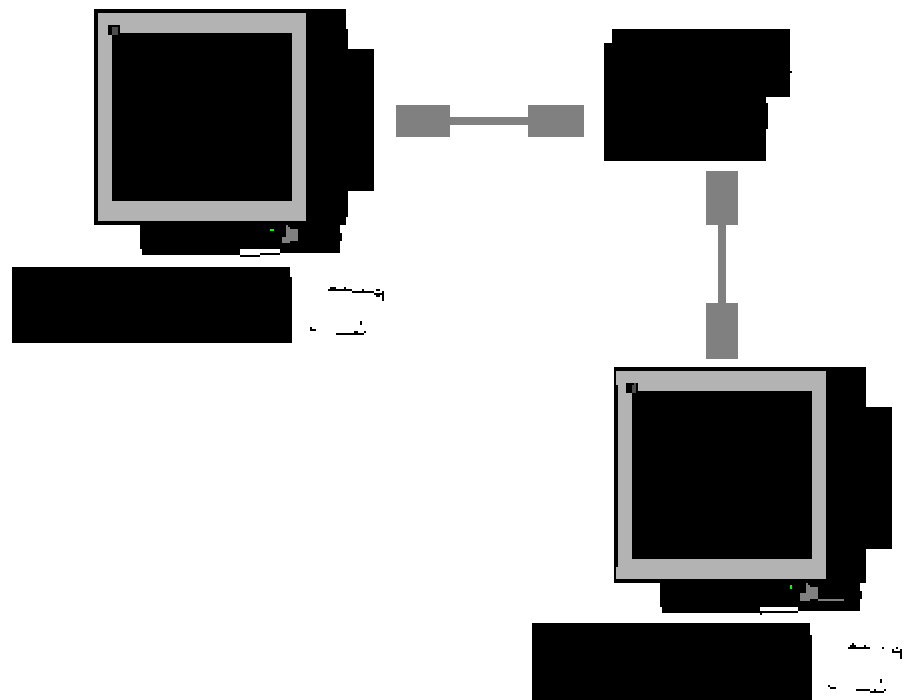
Paper automation, enabled by the advent of computers and laser printers, is the first stage of the document automation evolution. Most people think of the processing and assembly of business correspondence and forms by computers as document automation. While the computer does perform some information processing, this stage of document automation evolution is still very paper intensive. It does not extend to associated automated document workflow and procedures.



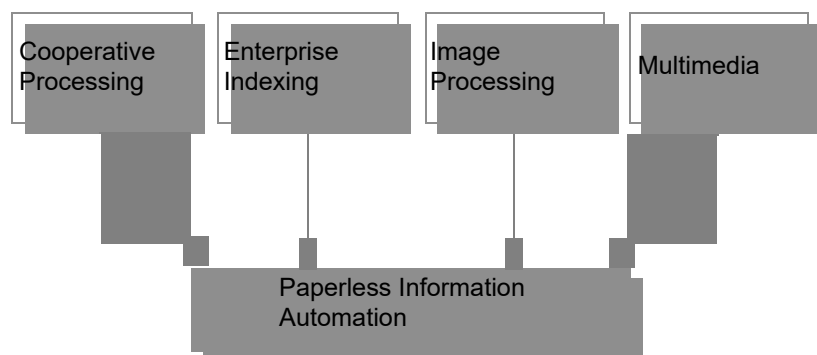
Stage 2 - workflow automation

Workflow automation, enabled by the proliferation of personal computers, communication standards, Local Area Networks (LANs), Wide Area Networks (WANs), and integrated FAX machines, is the second stage in the document automation evolution. Workflow automation goes beyond information processing to the transfer of digitized information across telecommunication lines. It eliminates many manual procedures, often clerical in nature, from the workflow process.

Stage 3 - paperless information automation



Paperless information automation combines multiple technologies across multiple organizations, enterprises, and government entities. Information elements from various sources are shared and are readily available in flexible electronic formats. Paperless information automation enables you to reuse the information contained in the documents. Electronic documents are much easier to track, maintain, update, route, file, and retrieve.



DOCUMENT AUTOMATION GOALS

Document automation combines many elements of the evolutionary stages previously discussed to accomplish these primary objectives:

- Eliminate paper

Paper consumes enormous resources. Document automation decreases the costs associated with paper documents, and decreases the requirements for both long term and short term storage, retrieval, and document distribution.

- Automate manual procedures

Automating manual procedures associated with document automation increases efficiency, increases accuracy, and reduces costs. Repetitive and unnecessary procedures are identified and eliminated.

- Automate system interfaces

Interfaces which allow exchange of data between automated systems eliminate the need to manually enter data. Automated system interfaces also eliminate the need to supplement automated processes with manual functions. Automated system interfaces reduce errors, increase efficiency, and simplify the workflow.

As you can see, document automation encompasses many different technologies which merge in a variety of ways. In the current business environment, there are many single technologies and partial solutions which mimic document automation at first glance. Keep in mind, a single solution using one technology is not document automation. Document automation involves multiple technologies which help you manage forms and documents, workflow, procedures, and other electronic media, based on the needs and requirements of each individual organization or enterprise.

SYSTEM BENEFITS

The system's cohesive design results in many benefits to the user. The system provides a seamless interface to your existing systems by integrating document automation technology with your current systems, and by offering you a customized computer system with reusable resources. You can select modules to meet your specifications.

The system also provides you with the following advantages in your document automation processing:

- **Functional** - The system's configuration meets a wide variety of document processing needs. The system's expandable architecture utilizes technological innovations to meet changing processing needs.
- **Portable** - The system's architecture allows core processing modules to operate on multiple hardware platforms and in multiple operating environments. This design gives the user control of the system configuration in order to meet individual needs.
- **Modular** - The system's configuration lets you select modules to customize your system. The modular design eases maintenance by segregating functions in independent modules. A change in one module does not necessitate multiple changes throughout the system. This modular design also improves performance by eliminating unnecessary processing.
- **Reusable** - The biggest advantage in using the system is the reusability of resources. Libraries are composed of customizable resource units such as images and rules, which can be reused. Reusing resources increases efficiency and promotes consistency throughout your system and product.
- **Easy to use** - System components have a graphical user interface common to all components. The system's seamless system interface provides transparent print and data merge capabilities.

Chapter 2

Understanding the System

In Chapter 1, you were introduced to the system as a whole. This chapter provides an overview of Documaker Server.

As you review this chapter you will learn about the programs that make up Documaker Server. Following the overview, you will learn about the files used and created by the system programs in both the multi- and single-step processes.

This chapter contains the following topics:

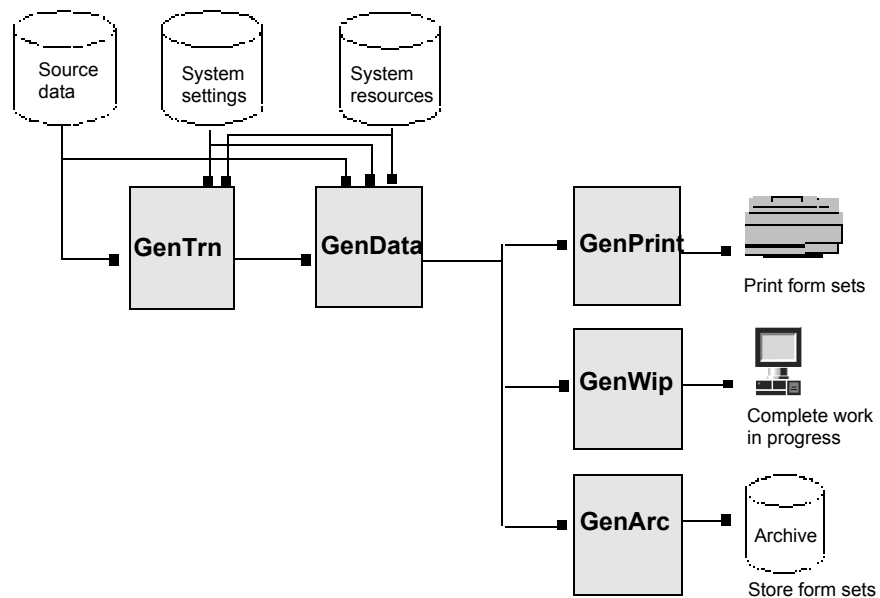
- [Processing Overview on page 10](#)
- [Processing Options on page 13](#)
- [Using Banner Processing on page 14](#)
- [Multi-step Processing on page 19](#)
- [Restarting the GenData Program on page 31](#)
- [Controlling GenTrn Processing on page 33](#)
- [Single-step Processing on page 35](#)
- [Using IDS to Run Documaker on page 47](#)
- [Writing Unique Data into Recipient Batch Records on page 48](#)
- [Running Documaker Using XML Job Tickets on page 57](#)
- [Handling 2-up Printing on page 58](#)
- [Controlling WIP Field Assignments on page 75](#)
- [Multi-mail Processing on page 78](#)
- [Adding and Removing Pages on page 81](#)
- [Adding Indexes and Tables of Contents on page 84](#)
- [Run-Time Options on page 85](#)
- [Using INI Built-In Functions on page 89](#)
- [Using XML Files on page 98](#)
- [Using XPath on page 100](#)

PROCESSING OVERVIEW

Documaker Servers designed to gather source data, process that data by applying rules you define, merge the data onto pre-designed forms, and print the result. In addition, Documaker Server can automatically check for incomplete data and send that data to Documaker for completion. Documaker Server can also automatically archive completed transactions which you can later view as needed.

The following illustration shows a high level view of Documaker Server:

NOTE This illustration and the other illustrations in this chapter show a typical, workstation-based system flow. Your system may be set up differently. Furthermore, the system can be customized in many ways and can run on a variety of platforms. For instance, if your source data is properly formatted, you can bypass the GenTrn program. Or, you may choose to run the GenTrn, GenData, and GenPrint programs on a host machine and then download the information and use a system utility (FIXOFFS) to prepare it for use by the GenWIP and GenArc programs running on a workstation. You could also run the GenArc program on the host and only run the GenWIP program on a workstation.



This illustration shows the main programs which make up Documaker Server and an overall view of the processing cycle.

- GenTrn. The GenTrn program reads source data and uses system settings to create transaction records. The source data is stored in extract files. Depending on the operating system you use, this program has various names such as *GENTNW32.EXE* for 32-bit Windows environments.
- GenData. The GenData program takes the transaction records created by the GenTrn program and uses system settings and resources to apply processing rules to those transactions.

The GenData program creates output files the GenPrint program can use. It also creates files with incomplete transactions which the GenWIP program can use. The GenWIP program creates from these files, output you can display and complete using the WIP module of Documaker Workstation.

The output from the GenData program is also used by the GenArc program to archive data. Depending on the operating system you use, this program has various names such as ~~GENDAW32.EXE~~ for 32-bit Windows environments.

NOTE The illustration on the preceding page and this overview discuss the standard or multi-step processing flow of the system. By using specific rules you can have the GenData program execute both the functions of GenTrn and GenPrint. This is called *single-step processing* and can improve performance. To learn more, see [Single-step Processing on page 35](#)

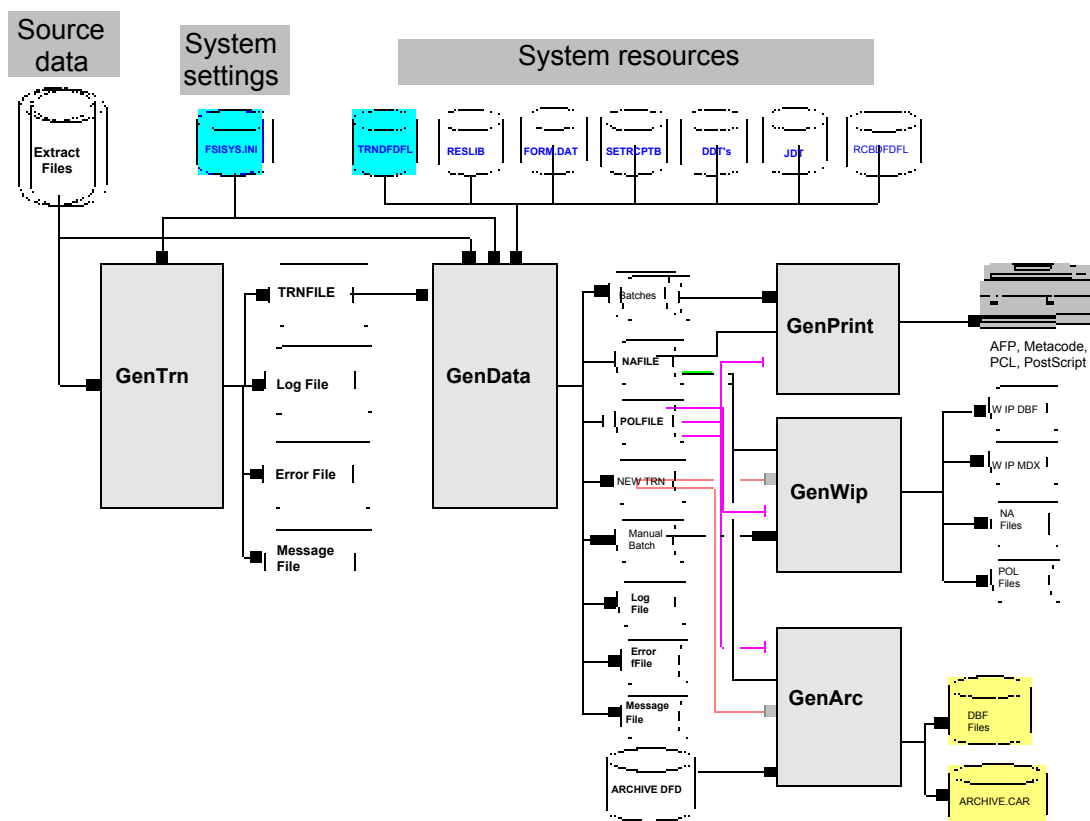
- GenPrint. The GenPrint program takes information produced by the GenData program and creates printer spool files for use with PCL, AFP, Metacode, and PostScript compatible printers. In addition, If you have purchased Skywire Software's Internet Document Server, the GenPrint program can also produce a Portable Document File or PDF (Acrobat) output. Depending on the operating system, this program has various names such as ~~GENPTW32.EXE~~ for 32-bit Windows environments.
- GenWIP. The GenWIP program receives information about incomplete transactions from the GenData program and processes that information so you can use the WIP module of Documaker to display the form and fill in the missing information. Once completed, you can print, archive, print and archive, delete, or change the status of form sets using Documaker. Depending on the operating system, this program has various names such as ~~GENWPW32.EXE~~ for 32-bit Windows environments.

NOTE When using Documaker Server, a transaction may be placed in WIP for completion by a data entry operator. In these cases, you would first complete the transaction before it is archived.

- GenArc. The GenArc program archives data so you can store the information efficiently and retrieve it quickly. This program receives information from the GenData program. Depending on the operating system, this program has various names such as ~~GENACW32.EXE~~ for 32-bit Windows environments.

The previous illustration showed a high level view of Documaker Server which shows you the main programs in the system and its processing cycle. These programs create and use several types of files as they process information. The following illustration shows this processing flow in greater detail, though not every possible system file is included.

Understanding how the information flows from one program to another and which files are used and created is key to understanding Documaker Server. Here you can see all of the files the system uses and creates during its processing cycle.



You can find information about all these files and programs in the Glossary. You can also find examples of certain files in Appendix [B System Files on page 389](#). Let's first look at the GenTrn program and the files it uses and creates.

PROCESSING OPTIONS

You can run Documaker Server as a multi- or single-step process. Variations of these processes provide additional options such as AFP 2-up printing and multi-mail sorting.

Chapter 2 begins with a general overview of the system. From this point forward, we will review specific processing options. The following topic discusses running the system using the multi-step process. This topic is followed by a discussion of running the system using the single-step processes. The remainder of the chapter provides brief explanations of 2-up and multi-mail printing.

NOTE To gain a complete understanding of the different features of the multi- and single-step processes, it is important to read through both sections. Certain information that is common to both processes is only described in the multi-step section.

To help determine which option is best suited for a particular need, a brief description of the run-time options and related processes are provided in the table below:

Process	Description
2-Up Printing	Two-up printing is a two-step process which passes input through GenData three (3) times with a different JDT file each pass. This process is similar to the single-step process in that GenData performs the work, but the three passes through GenData actually represent two steps of the multi-step process: processing the transactions and printing the transactions. Two-up printing is AFP printer-specific. For more information, see Handling 2-up Printing on page 58
Banner	The system lets you process banners at several points in the processing cycle. Doing this involves using a simplified AFGJOB.JDT file. For more information, see Using Banner Processing on page 14
Multi-mail	GenData groups transactions with the same multi-mail code into selected print batches to be sorted and delivered to the same location. For more information, see Multi-mail Processing on page 78
Multi-step	The system programs, GenTrn, GenData and GenPrint, each perform a set of steps to read data, create output files and print. GenWIP and GenArc are optional programs to complete incomplete transactions and archive data for retrieval. For more information, see Multi-step Processing on page 19
Restarting the system	You can set up the GenData program to restart itself at a particular transaction if it encounters a failure. For more information, see Restarting the GenData Program on page 31
Single-step	To enhance system performance, the steps of the GenTrn, GenData and GenPrint programs are performed in one step by GenData. The GenWIP and GenArc programs function the same as in the multi-step process. For more information, see Single-step Processing on page 35

USING BANNER PROCESSING

The system includes support for banner processing. Banner processing is supported at these points in the processing cycle:

- Beginning of a batch
 - Before a transaction is processed
 - After a transaction is processed
- End of a batch

Banner processing is optional at each point. Banner processing can optionally include FAP forms processing and DAL script processing.

You specify the FAP forms for banner processing in this manner:

```
;key1;key2;form name;
```

The forms must appear in the FORM.DAT file in DefLib. The associated images for those forms must reside in FormLib.

You can set up banner forms and scripts at a global level so they can be used by all print batches. Individual recipient print batches can specify local forms or scripts to override the global forms and scripts.

Keep in mind these limitations:

- This enhancement only affects the GenPrint program. Documaker Workstation has a separate banner handling method, and does not support this method of banner processing.
- Only the standard printer drivers, such as AFP, Metacode, PCL, and Postscript, support batch banner processing. Avoid batch banner processing if you are using another print driver.
- Banner pages are printed at the group level. As a result, this bypasses the custom callback function named in the CallbackFunc option of the Print control group since it is a form set-level callback.

NOTE Version 10.1 added batch-level banner processing to multi-step mode. Version 10.2 added batch-level banner processing to single-step processing — printing via GenData using the PrintFormset rule.

Banner processing is not recommended for use with multifile print, since banner pages are not applicable. It is, however, still supported since you may have reasons to gain control and execute DAL scripts before or after batches or transactions are processed.

Enabling banner processing

For performance reasons banner processing is, by default, disabled. You must enable it using one or both of these INI options:

```
< Printer >  
EnableTransBanner = True  
EnableBatchBanner = True
```

Omitting either option disables the associated level of batch banner processing. Once enabled, banner processing is in effect for the entire GenPrint run. You can, however, disable banner processing for individual batches by specifying forms and scripts with blank names.

Specifying banner forms and scripts

You can globally specify forms and scripts for all batches, or locally for specific batches. Use these INI options to specify global batch forms and scripts:

```
< Printer >
  BatchBannerBeginForm = form name
  BatchBannerBeginScript = script name
  BatchBannerEndForm    = form name
  BatchBannerEndScript  = script name
  TransBannerBeginForm  = form name
  TransBannerBeginScript = script name
  TransBannerEndForm    = form name
  TransBannerEndScript  = script name
```

Specify form names as follows:

```
;KEY1;KEY2;Form name;
```

You must have an associated form line in the FORM.DAT file to match the specified form. The images (FAP files) for the forms are specified in the form lines in the FORM.DAT file. You must include these FAP files in FormLib.

Store the banner forms in a separate and unique banner form group, defined by a combination of *Key1* and *Key2*. You can use the AddForm DAL function in a DAL script to insert additional forms for banner processing. Place these additional forms and images in the same group as the initial banner form. Each form is printed separately and after all banner forms are printed, the entire banner group is removed from the document set. For these reasons, it is critical that you isolate the banner forms from the rest of the transaction document set by specifying *Key1/Key2* combination that does not otherwise occur within the document.

The FAP files assigned to the form (on the form line in the FORM.DAT file) must have the recipient **BANNER** with a copy count of at least one. When banner forms are printed, only images assigned to the recipient BANNER with a non-zero copy count are printed.

Specify the DAL script names without a path or extension. For best results, store the DAL scripts in your DAL libraries because they are easier to maintain. The system automatically loads DAL libraries if you include these INI options:

```
< DALLibraries >
  LIB = library1
  LIB = library2
```

The DAL script libraries or files must reside in DefLib.

You can specify forms and scripts at the recipient batch level to override the global specification. Here is an example of how you do this:

```
< Print_Batches >
  BATCH1 = BATCH1.BCH
  BATCH2 = BATCH2.BCH
< Batch1 >
  BatchBannerBeginForm = form name
  BatchBannerBeginScript = script name
  BatchBannerEndForm    = form name
  BatchBannerEndScript  = script name
  TransBannerBeginForm  = form name
  TransBannerBeginScript = script name
  TransBannerEndForm    = form name
  TransBannerEndScript  = script name
```

You can specify some, none, or all of the forms and scripts for local override of the default global forms and scripts.

An individual batch can completely or partially disable banner processing if the forms, script names, or both are blank, as shown here:

```
< Batch1 >
  BatchBannerBeginForm =
  BatchBannerBeginScript =
  BatchBannerEndForm    =
  BatchBannerEndScript =
  TransBannerBeginForm =
  TransBannerBeginScript =
  TransBannerEndForm    =
  TransBannerEndScript =
```

Processing logic

Banner processing functions are part of the base system and are primarily located in GenLib. The GenPrint program, however, first routes the processing to CusLib. This lets you use the exit points in CusLib to create additional customized processing before, after, or in place of, the calls to GenLib routines.

The processing sequence for banner processing (at any level) is as follows:

- 1 If a banner form is specified, it is created in the form set and the FAP files are loaded.
- 2 If a banner DAL script is specified, it is executed.
- 3 For any banner form specified in step 1 or created during step 2, the following steps take place:
 - any variable fields in the banner form that are still empty are updated, first from matching GVM variables, such as fields in the recipient batch record, then from matching DAL variables.
 - the form is printed.
- 4 If there were banner forms to process, after updating the fields and printing the forms, the entire banner form group is removed from the form set.

NOTE You can suppress the printing of the banner page by using the SuppressBanner DAL function. This is useful when you need to combine several transactions within the same transaction banner pages.

If there are registered *comment record* functions, each banner form in the form group receives its own set of comment records. If the additional forms should not receive their own comment records, add the images for those forms to the original form—do not add them as separate forms.

DAL functions You can also use these DAL functions with banner processing. See [DAL Reference](#) for more information.

- **RecipName.** Returns the name, such as INSURED, AGENT, COMPANY, and so on, of the recipient batch record of the transaction currently being printed.
- **RecipBatch.** Returns the name, such as BATCH1, BATCH2, ERROR, MANUAL, and so on, of the recipient batch file being processed.
- **SuppressBanner.** Suppresses the current banner from printing. You can use this function when you want to combine several transactions inside one set of banner pages, based on a flag that the DAL script checks.

Banner processing example

Assume you have these FAP files in your forms library (FormLib).

- btchbannr
- btctrail
- trnbannr
- trntrail

Here is an excerpt from the FSISYS.INI file:

```
< Printer >
  PrtType = PCL
  EnableTransBanner = TRUE
  EnableBatchBanner = TRUE
  BatchBannerBeginScript = PreBatch
  TransBannerBeginScript = PreTrans
  BatchBannerEndScript = PstBatch
  TransBannerEndScript = PstTrans
  BatchBannerBeginForm = ;BANNER;BATCH;BATCH BANNER;
  BatchBannerEndForm    = ;BANNER;BATCH;BATCH TRAILER;
  TransBannerBeginForm = ;BANNER;TRANSACTION;TRANS HEADER;
  TransBannerEndForm    = ;BANNER;TRANSACTION;TRANS TRAILER;
< DALLibraries >
  LIB = Banner
```

Here is an excerpt from the FORM.DAT file:

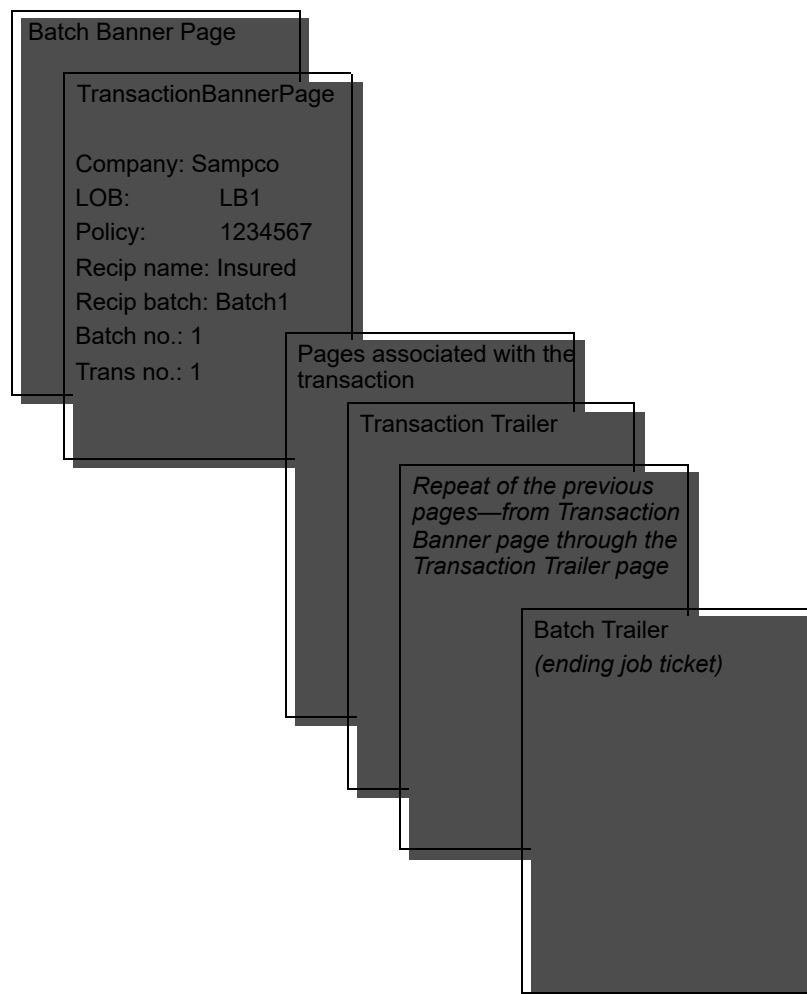
```
;BANNER;BATCH;Batch Banner;Batch Banner (Job\
Ticker);N;;btcbannr|D<BANNER(1)>;
;BANNER;BATCH;Batch Trailer;Batch Trailer (End\
Ticket);N;;btctrail|<BANNER(1)>;
;BANNER;TRANSACTION;Trans Trailer;Transaction Trailer (End\
Ticket);N;;trntrail|D<BANNER(1)>;
;BANNER;TRANSACTION;Trans Header;Transaction Banner\
Page;N;;trnbannr|D<BANNER(1)>;
```

Here is an example of the BANNER.DAL file in DefLib:

```
BeginSub PreBatch
    #batch += 1
    #trans = 0
    rb = RecipBatch()
    rn = RecipName()
EndSub

BeginSub PreTrans
    #trans += 1
    rb = RecipBatch()
    rn = RecipName()
EndSub
```

These additions to the FORM.DAT and FSISYS.INI files plus file additions to the FormLib and DefLib sub-directory would cause the following pages to be added to each batch:



MULTI-STEP PROCESSING

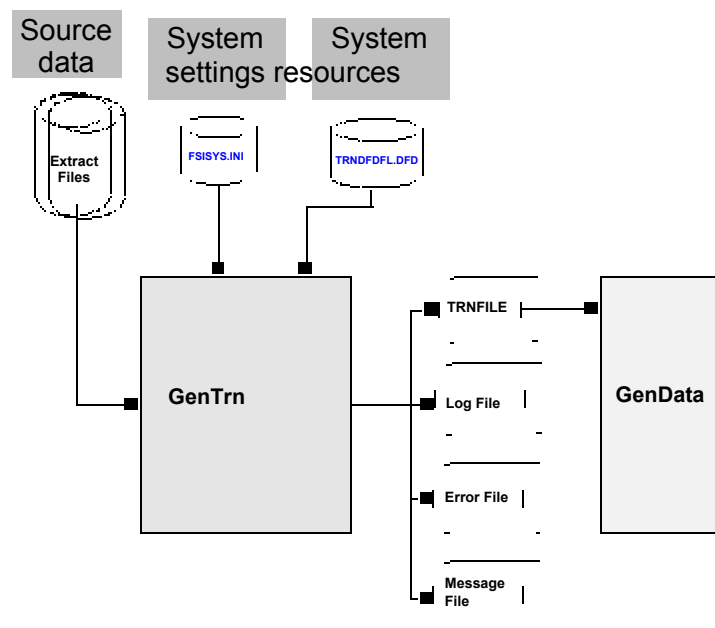
This topic describes the standard, multi-step approach to processing. In a multi-step processing scenario, the system takes these steps:

- Create the transaction records
- Process the transactions
- Create print spool files
- Send incomplete transactions to work-in-progress (WIP)
- Archive transactions

NOTE Be sure to carefully read this topic even if you are using single-step processing.

CREATING TRANSACTION RECORDS

This illustration shows the files used and created by the GenTrn program as it creates transaction records:



The GenTrn program takes the source data, which is stored in extract files, and creates a list of the transactions, which is stored in the TRNFILE, or transaction file. This transaction list is then used by the GenData program as it processes the transactions.

The GenTrn program uses settings in the FSISYS.INI and TRNDFDFL.DFD files to determine how to process the transactions. These files provide the GenTrn program with information about the format and structure of the extract file, such as how to determine where each new record starts.

The GenTrn program also produces a log file of its activities, a message file, and an error file which you can use to resolve any errors that occur.

File Summary

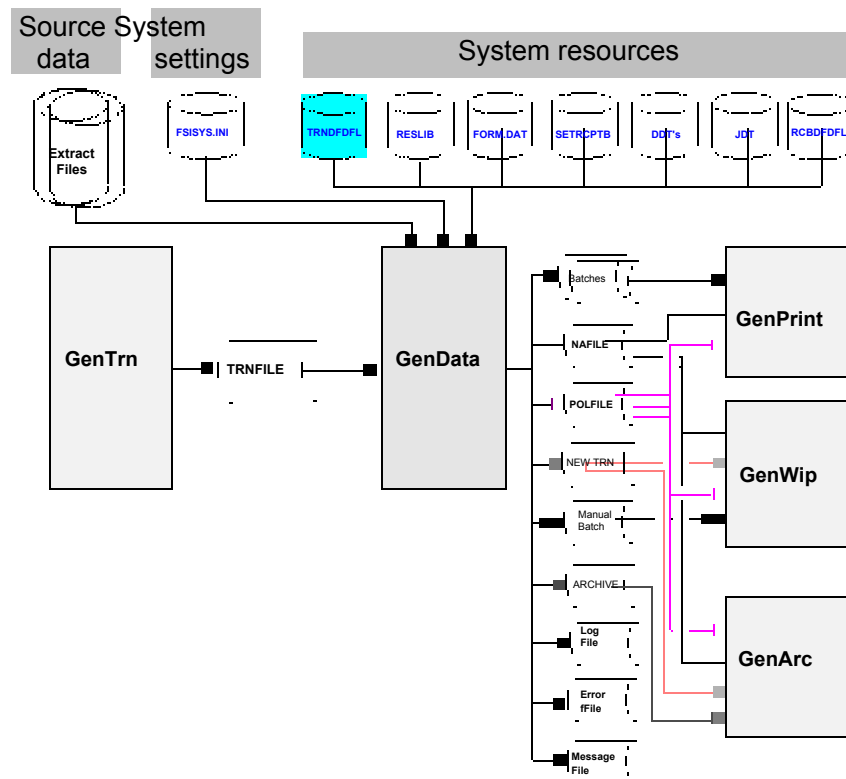
This table summarizes the files used to supply information (input) and the files created by (output) the GenTrn program:

NOTE You can use the Data control group in the FSISYS.INI file to specify the names and extensions for all other input and all of the output files.

	File Name or Type	Default Extension	File Format	Description
Input	Extract files	.DAT	text	Contains the data you want to process.
	FSISYS	.INI	text	Initialization file which includes system settings.
	TRNDFDFL	.DFD	text	Defines the attributes of the variable fields in the TRNFILE.DAT file.
Output	TRNFILE	.DAT	text	Serves as an index to the individual transactions. Used by the GenData program as it processes the source data in the extract file.
	Log file	.DAT	text	Serves as a processing log for the GenTrn program. The system records the information by transaction.
	Error file	.DAT	text	Notes any errors and warnings encountered by the GenTrn program as it created the TRNFILE.DAT file. The system records the information by transaction.
	Message file	.DAT	text	Contains errors and warnings.

PROCESSING TRANSACTIONS

The following illustration shows the files used and created by the GenData program as it processes transactions:



The GenData program uses the transaction list (TRNFILE) created by the GenTrn program as it processes the source data stored in the extract files. The FSISYS.INI file provides system setting information, such as whether or not it should stop processing if it encounters errors, how to identify key fields in extract files, whether or not it should check the output data size against the defined field length, and so on.

The files listed under *System resources* provide additional information such as:

- How to read the transaction file (TRNDFDFL.DFD)
- The forms, logos, and other resources to use when creating the form sets (RESLIB)
- What forms to use (FORM.DAT)
- Who to send the forms to (SETRCPTB.DAT)
- What processing rules to apply to the data (DDTs)
- What processing rules to apply to this job (JDTs)
- How the batch files are defined (RCBDFDFL.DFD)

NOTE You can learn more about these files in Appendix B, [System Files on page 389](#)

Output Files for GenPrint

The output files created by the GenData program include three types of files used by the GenPrint program: Batch files, NAFILES, and POLFILES. Batch files list the transactions which should be included in each batch print job. NAFILES store image and variable field information. POLFILES define the form set the GenPrint program should use for each transaction it processes.

Output Files for GenWIP

The GenWIP program also uses the NAFILE and POLFILE to store image and variable field information and to define the form sets. In addition, the GenData program creates manual batch files specifically for the GenWIP program.

The GenData program creates manual batch files if it is unable to complete the processing of a form set. Typically, this occurs if the form set is missing information. The GenWIP program uses this file to create separate transactions which can then be completed manually using the Entry module of Documaker Workstation. The data for the separate transactions are stored in files with the extension **DAT**, such as 00000001.DAT, 00000002.DAT, and so on.

Output Files for GenArc

The GenArc program also uses the NAFILE and POLFILE to store image and variable field information and to define the form sets. In addition, the GenArc program uses the NEWTRN files to tell it where to find data in the NAFILES and which forms to use in the POLFILES.

File Summary

This table summarizes the files used to supply information (input) and the files created by (output) the GenData program:

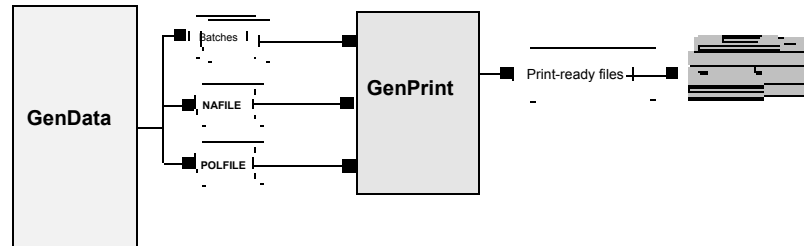
NOTE You can use the Data control group in the FSISYS.INI file to specify the names and extensions for all other input and all of the output files.

	File name or Type	Default Extension	File Format	Description
Input	Extract files		text	Contains the data you want to process.
	FSISYS	INI	text	Initialization file which includes system settings.
	TRNFILE	DAT	text	Used as an index to the individual transactions stored in the extract file.
	TRNDFDFL	DFD	text	Tells GenData how to read the TRNFILE.
	FORM	DAT	text	Defines the forms in a form set.
	SETRCPTB	DAT	text	Defines the recipients of a form set.
	DDT files	DDT	text	Contains the rules GenData applies to the data.
	JDT files	JDT	text	Contains the rules GenData follows when processing the job.
	RCBDFDFL	DFD	text	Defines the attributes of the variable fields in a batch file.
	Resources	(various)	(various)	Includes logos (.LOG), font cross reference files (.FXR), images (.FAP), and so on.
Output	Batch files	BCH	text	Indicates which transactions should be included in a given batch job. Used by the GenPrint program.
	NAFILE	DAT	text	Contains image and variable field information. Used by the GenPrint, GenWIP, and GenArc programs.
	POLFILE	DAT	text	Defines the forms to use for each batch. Used by the GenPrint, GenWIP, and GenArc programs.
	NEWTRN	DAT	text	Tells the GenArc program where to find data in the NAFILE and which forms to use in the POLFILE.

File name or Type	Default Extension	File Format	Description
Manual batch files	BCH	text	Created if the form is incomplete. Used by GenWIP to allow an operator to complete the form in the Entry module of Documaker.
Error batch files	.BCH	text	Created if the system spots an error, such as if the system spots an error and the form is marked as host required. In contrast to manual batch files, you cannot correct these errors using the GenWIP program. Instead, you must correct the error in the extract file, change the flag to operator required, or change the FAP file and then process the transaction again.
ARCHIVE	DFD	text	Tells the GenArc program how to store archived data.
Log file	DAT	text	Serves as a processing log. Created by the GenTrn program, the GenData program adds information to this file.
Error file	DAT	text	Notes any errors encountered by the GenData program. Created by the GenTrn program, the GenData program adds information to this file (as do the GenPrint, GenWIP, and GenArc programs).
Message file	.DAT	text	Contains errors and warnings.

CREATING PRINT SPOOL FILES

The following illustration shows the files used and created by the GenPrint program as it creates print-ready files:



The GenPrint program receives batch files from the GenData program which tell it what transactions to print, NAFILES which tell it what data to print, and POLFILES which tell it which forms to print.

With this information, the GenPrint program creates print-ready files for AFP, Xerox Metacode, PCL, or PostScript compatible printers. The GenPrint program serves as the print engine for the system.

NOTE In addition, the GenPrint program can also create PDF (Acrobat) if you have purchased Skywire Software's PDF Print Driver. For more information about this product, contact your sales representative.

File Summary

This table summarizes the files used to supply information (input) and the files created by (output) the GenPrint program:

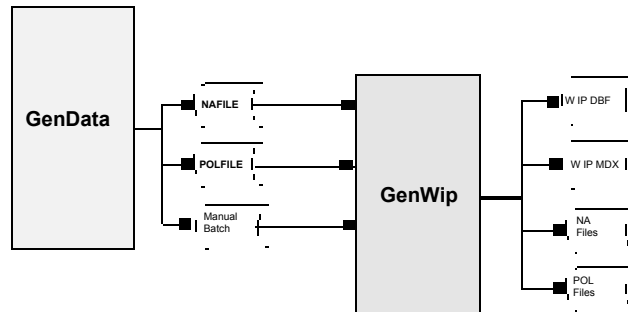
NOTE You can use the Data control group in the FSISYS.INI file to specify the names and extensions for all other input and all of the output files.

	File name or Type	Default Extension	File Format	Description
Input	Batch files	BCH	text	Indicates which transactions should be printed in a given batch. Used as trigger files by the GenPrint program.
	NAFILE	DAT	text	Contains image and variable field information.
	POLFILE	DAT	text	Defines the forms to use for each batch.
	RCBDFDFL	DFD	text	Defines the attributes of the variable fields in a batch file.
Output	Print-ready AFP, PCL, files	XER, PST, PDF*	AFP, PCL, MetaCode, PostScript, or PDF*	Printer spool files which can be printed on the printer of your choice.

* To produce PDF files, you must also have Skywire Software's Internet Document Server.

SENDING INCOMPLETE TRANSACTIONS TO WIP

The following illustration shows the files used and created by the GenWIP program as it processes incomplete transactions:



The GenWIP program receives information from the GenData program about incomplete transactions the GenData program found during its processing cycle. With this information, the GenWIP program creates files the WIP module of Documaker can read. Through the WIP module, data entry operators can complete the transactions by entering the missing information.

The manual batch file tells the GenWIP program which transactions are incomplete and should be included in work-in-progress (WIP).

Using the information in the manual batch files, the GenWIP program extracts the information it needs from the NAFILE and POLFILE. With this information, it then creates individual NA and POL files for each incomplete transaction. The GenWIP also creates a WIP.DBF (database) file which contains information about the incomplete transactions. The WIP.MDX file serves as an index to this file. Both the WIP.DBF and WIP.MDX files are used by the WIP module of Documaker Workstation.

File Summary

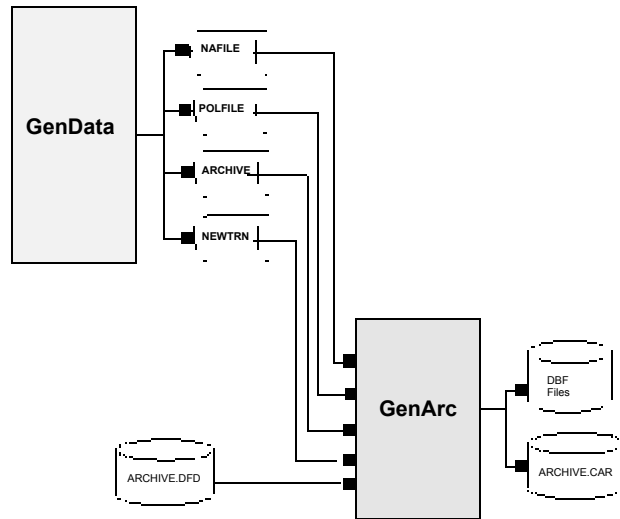
This table summarizes the files used to supply information (input) and the files created by (output) the GenWIP program:

NOTE You can use the Data control group in the FSISYS.INI file to specify the names and extensions for all other input and all of the output files.

	File Name or Type	Default Extension	File Format	Description
Input	NAFILE	DAT	text	Contains image and variable field information.
	POLFILE	DAT	text	Defines the forms to use for each batch.
	RCBDFDFL	DFD	text	Defines the attributes of the variable fields in the batch files.
	Manual batch	BCH	text	Indicates which transactions should be included.
Output	WIP	DBF		Contains information about the incomplete transactions extracted from the NAFILE and POLFILE.
	WIP	MDX		Serves as an index to the WIP.DBF file.
	NA Files	DAT	text	Contains the data (image and variable field information) for a specific transaction. These files are named numerically and each file has a corresponding POL file.
	POL Files	POL	text	Defines the forms to use for a specific transaction. These files are named numerically and each file has a corresponding NA file.

ARCHIVING TRANSACTIONS

The following illustration shows the files used and created by the GenArc program as it archives completed transactions:



The GenArc program receives information from the GenData program, using many of the same files used by the GenWIP and GenPrint programs, such as the NAFILE and POLFILE. These two files identify the data to archive. The NEWTRN file tells the GenArc program where to find data in the NAFILE, which is created by the GenArc program.

In addition, the GenArc program also uses the ARCHIVE.DFD file which tells it how to store the data.

With this information, the GenArc program creates DBF and CAR files. The DBF files serve as an index to the CAR files, where the archived information is actually stored. You can have multiple CAR files.

File Summary

This table summarizes the files used to supply information (input) and the files created by (output) the GenArc program:

NOTE You can use the Data control group in the FSISYS.INI file to specify the names and extensions for all other input and all of the output files.

	FileName or Type	Default Extension	File Format	Description
Input	NAFILE	DAT	text	Contains image and variable field information.
	POLFILE	DAT	text	Defines the forms to use for each batch.
	NEWTRN	DAT	text	Tells the GenArc program where to find data in the NAFILE and which forms to use in the POLFILE.
	APPIDX	DFD	text	Tells the GenArc program how to store the data.
Output	DBF files	DBF	text	Serves as an index to the archived data in the CAR files.
	ARCHIVE	CAR	CAR	Contains the archived forms.

RULES USED IN MULTI-STEP PROCESSING

Several rules are used to execute the programs of the multi-step process. For a complete listing and description of these and other rules, see the [Rules Reference](#).

RESTARTING THE GENDATA PROGRAM

You can set up the GenData program to restart itself at a particular transaction if it encounters a failure. To accomplish this, the system uses a restart file. You use INI options to set up the restart file.

NOTE This feature does not apply if you are using single-step processing.

The restart file stores checkpoint information at specified intervals. If an error is encountered, the program resets itself and then checks each transaction until it isolates the transaction causing the error.

The restart file is removed at the end of a successful run. If the file exists at the start of a GenData run, the system assumes a restart is necessary and will open and read the file. The checkpoint information lets the system set internal pointers and output files in such a way that it can begin at that transaction.

These rules are used to handle restarting the GenData program:

- RULCheckTransaction
- RestartJob

RULCheckTransaction rule

The RULCheckTransaction rule is always the first base form set rule. It saves the EXTRFILE offset, TRNFILE offset, NEWTRN offset, NAFILE offset, POLFILE offset, and batch file offsets into a restart (RSTFILE) file.

These offsets are updated in the post process after a specific number of transactions. You specify the number of transactions using the CheckCount option. You define the Restart file and the and check count in the Restart control group:

```
< Restart >
  RstFile =
  CheckCount =
```

Option	Description
RstFile	Enter the name of the restart file. If you omit this option, the system uses RSTFILE.RST (DD:RSTFILE for MVS) as the file name. The system uses the DataPath option in the Data control group to determine where to create the restart file. The default location is the current working directory.
CheckCount	Enter a number to specify the number of transactions to process before updating the offsets. For instance, if you specify two hundred (200), the system processes two hundred transactions, updates the offsets, processes two hundred more transactions, and so on. The default is 100. You can also use the <code>cnt</code> command line option with the GenData program to override the CheckCount option. Here is an example: <code>gendaw32 /cnt=10</code>

Here is an example:

```
;RULCheckTransaction;2;Always the first form set rule;
```

RestartJob rule The RestartJob is always the first base rule. This rule opens the restart file (RSTFILE) and resets the EXTRFILE, TRNFILE, NEWTRN, NAFILE, POLFILE, and batch files at the broken transaction if the restart file exists. If the restart file does not exist, the RestartJob rule is skipped.

NOTE For more information on these rules, see the [Rules Reference](#). You can also set up the GenArc program to restart itself. For more information, see [Setting the Restart Option on page 333](#).

Here is an example:

```
;RestartJob;1;Always the first base rule;
```

INI options To use the restart feature, you should also set the following INI options:

```
< GenDataStopOn >
BaseErrors          = Yes
TransactionErrors   = Yes
ImageErrors         = Yes
FieldErrors         = Yes
```

CONTROLLING GENTRN PROCESSING

Include the following control group and option in the FSISYS.INI file when you want the GenTrn program to continue processing transactions when errors occur. By default, the GenTrn program halts when it encounters an error.

NOTE This control group and option is typically used if you are using XML extract files and you do not want the GenTrn program to stop every time it encounters an error. For any type of extract file, using this option detects missing Key1, Key2, or Key3 information.

Here is an example of the control group and option:

```
< GenTranStopOn >
  TransactionErrors = Parameter1;Parameter2;Parameter3;
```

Parameter	Description
Parameter1	Enter No to turn the GenTranStopOn option off. The default is Yes.
Parameter2	Enter the name of the transaction file. To write out the error transaction, enter the name of the file where you want the extract file records written. If you omit the path, the system uses the DataPath option in the Data control group in the FSISYS.INI file to determine where to locate this file.
Parameter3	The system only looks at this parameter if you entered a file name for Parameter2. Enter Yes to tell the system to append the error transactions accumulated during this processing run to the file created in a prior run. Enter No to tell the system to overwrite any existing file. If Parameter2 exists and you omit this parameter, the system defaults to No. If you enter Yes, you must remove the file when necessary. Keep in mind that over a series of processing runs, this file will expand in size.

Separate the parameters with semicolons (;).

The system records all errors and warnings it encounters during a processing run in the ERRORFILE.DAT file. In addition, it writes the extract file records of the transaction in error to the file you specify in Parameter2. This lets you inspect those transactions and determine the best way to proceed.

Here are some examples. This option:

```
TransactionErrors = No;..\Extracts\ErrorTransaction.dat;No;
```

Is the same as:

```
TransactionErrors = No;..\Extracts\ErrorTransaction.dat;;
```

Both let the GenTrn program continue processing subsequent transactions when errors occur. These options tell the GenTrn program to write the error transaction to a file named ERRORTRANSACTION.DAT, stored in the \Extracts directory.

```
TransactionErrors = No; ErrorTransaction.dat;Yes;
```

This option lets the GenTrn program continue processing subsequent transactions when errors occur. Since the path of the error transaction file was omitted, the system uses the DataPath option in the Data control group in the FSISYS.INI file to find the file so it can append any error transactions to the existing error transaction file.

```
TransactionErrors = No;;
```

This option lets the GenTrn program continue processing subsequent transactions when errors occur. It does not, however, write out error transactions.

When using this option, you may encounter these errors:

- Problem in loading the XML file. Syntax error.

```
GenTrn
```

```
Transaction Error Report - System timestamp: Mon Dec 16 13:42:27 2002
DM12041: Error: FAP library error: Transaction:<111111111>,
area:<DXMLoadXMLRecs>
  code1:<48>, code2:<0>
  msg:<XML Parse Error: The 15 chars before error=< <Key1>Comp1<>,
the 8 chars starting at error=</Key1c>
>>.
DM12041: Error : FAP library error: Transaction:<111111111>,
area:<DXMLoadXMLRecs>
  code1:<48>, code2:<0>
  msg:<mismatched tag at line 3 column 16>.
DM10293: Error: Error in <BuildTrnRecs>: Unable to
<DXMLoadXMLRecs()>.
  Skip Transaction# <2>.
Warning: the specific info you see may not be the info for the error
transaction. It may be the info on the last complete transaction.

==> Warning count:    0
==> Error count:      3
```

- No problem in loading the file, however, Key1 is omitted in the transaction.

```
GenTrn
```

```
Transaction Error Report - System timestamp: Fri Dec 13 13:52:13 2002
DM1002: Error: Required INI definition omitted.
Cannot locate INI group <Key1Table> with value = defined.
DM15062: Error in BuildTrnRecs(): Unable to GENGetDocSetNames(pRPS).
Skip Transaction# <3>.

==> Warning count:    0
==> Error count:      2
```

SINGLESTEP PROCESSING

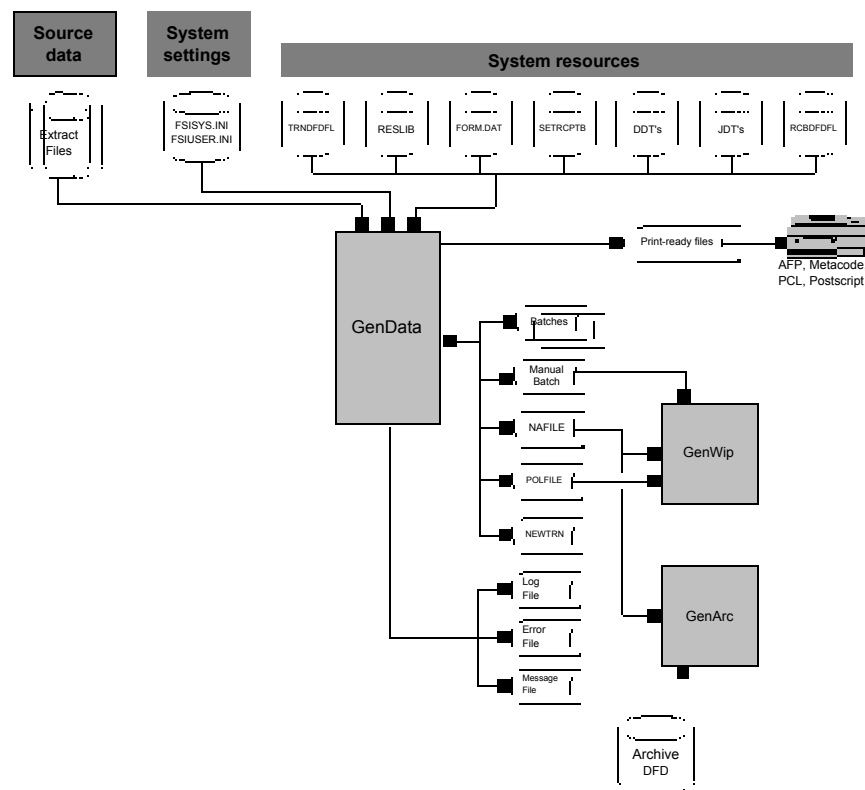
The single-step process improves the performance of your system by combining the functions of GenTrn, GenData and GenPrint into one step performed by GenData. This process is used when no intermediate steps are necessary.

The GenWIP and GenArc options are performed the same as in the multi-step process. See [Sending Incomplete Transactions to WIP on page 27](#) and [Archiving Transactions on page 29](#) for more information on the functions of the GenWIP and GenArc programs.

NOTE When running in single-step mode, you can only produce a single print stream. For instance, the most common method of print batching is to batch by recipient, in single-step processing, however, you cannot produce separate print streams for each recipient batch.

CREATING AND PROCESSING TRANSACTION RECORDS

In the multi-step process, the GenTrn program creates transaction records that are sent to the GenData program for processing. In the single-step process, the GenData program performs both of these actions in one step.



As shown in the illustration above, the GenData program processes transaction records, originated from the source data, and creates various output files for print, WIP or GenArc. By combining the functions of GenTrn and GenPrint into GenData, you reduce the number of times the system needs to open and close files, thus enhancing the overall performance of your system.

System Settings and Resources

The FSISYS.INI and the FSIUSER.INI file provide system setting information, such as whether or not it should stop processing if it encounters errors, how to identify key fields in extract files, whether or not it should check the output data size against the defined field length, and so on.

The files listed under system resources provide additional information such as:

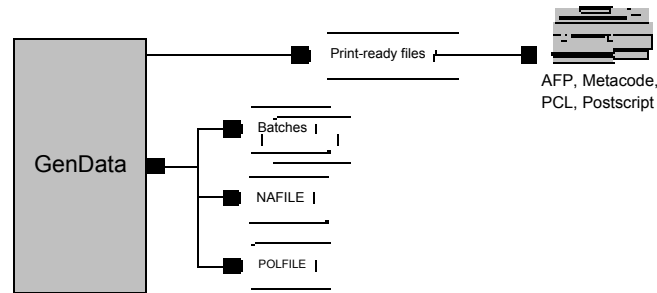
- How to read the transaction file (TRNDFDFL.DFD)
- The forms, logos, and other resources to use when creating the form sets (RESLIB)
- What forms to use (FORM.DAT)
- Who to send the forms to (SETRCPTB.DAT)
- What processing rules to apply to the data (DDTs)
- What processing rules to apply to this job (JDTs)
- How the batch files are defined (RCBDFDFL.DFD)

NOTE You can learn more about these files in Appendix B, [System Files on page 389](#)

The advantage of single-step processing is the improvement to performance. The disadvantage is that it is much more difficult to correct errors because the system does not create batch files at the end of each step. These batch files tell you what occurred and help you spot and correct errors.

CREATING PRINT FILES

With the placement of specific rules, you can make the GenData program perform the functions of the GenTrn and GenPrint programs. In other words, when GenData is processing transactions files, it is also producing the print-ready files necessary to print on AFP, Metacode, PCL, or Postscript printers.



As in the multi-step process, the GenData program creates these types of files:

- Batch files - list the transactions which should be included in each batch print job
- NAFILES - store image and variable field information
- POLFILES - define the form set the GenPrint program should use for each transaction it processes

NOTE When using single-step processing, you should clear all messages before each processing run. For information on how to do this, see [Clearing Messages on page 308](#)

File Summary

This table summarizes the files used to supply information (input) and the files created by (output) the GenData program:

NOTE You can use the Data control group in the FSISYS.INI file to specify the names and extensions for all other input files and all of the output files.

	File name or Type	Default Extension	File Format	Description
Input	Extract files		text	Contains the data you want to process.
	FSISYS	INI	text	Initialization file which includes system settings.
	TRNDFDFL	DFD	text	Tells GenData how to read and write the TRNFILE.
	FORM	DAT	text	Defines the forms in a form set.
	SETRCPTB	DAT	text	Defines the recipients of a form set.
	DDT files	DDT	text	Contains the rules GenData applies to the data.
	JDT files	JDT	text	Contains the rules GenData follows when processing the job.
	RCBDFDFL	DFD	text	Defines the attributes of the variable fields in a batch file.
	Resources	(various)	(various)	Includes logos (LOG), font cross reference files (FXR), images (FAP), and so on.
Output	Batch files	BCH	text	Indicates which transactions should be included in a given batch job.
	NAFILE	DAT	text	Contains image and variable field information. Used by the, GenWIP, and GenArc programs.
	POLFILE	DAT	text	Defines the forms to use for each batch. Used by the GenWIP and GenArc programs.
	NEWTRN	DAT	text	Tells the GenArc program where to find data in the NAFILE and which forms to use in the POLFILE.
	Manual batch files	BCH	text	Created if the form is incomplete. Used by GenWIP to allow an operator to complete the form in the Entry module.

File name or Type	Default Extension	File Format	Description
Error batch files	.BCH	text	Created if the system spots an error, such as if the system spots an error and the form is marked as host required. In contrast to manual batch files, you cannot correct these errors using the GenWIP program. Instead, you must correct the error in the extract file, change the flag to operator required, or change the FAP file and then process the transaction again.
ARCHIVE	DFD	text	Tells the GenArc program how to store archived data.
Log file	DAT	text	Serves as a processing log. Created by the GenData program in the single-step process.
Error file	DAT	text	Notes any errors encountered by the GenData program. Created by the GenData program in the single-step process.
Message file	.DAT	text	Intermediate file which contains log and error messages. These messages are then translated and written to either the LOGFILE.DAT or ERRFILE.DAT files.

MultiFilePrint callback function

The system includes a MultiFilePrint callback function designed for running the GenData program in single-step mode. The log file is either a semicolon delimited text file—the same as the file created by MultiFilePrint—or an XML file.

The layout of the XML file is as follows:

```
-
-
.\data\BATCH1.BCH
SAMPCO
LB1
1234567
T1
INSUREDS COPY
DATA\0rDcP7WxytE82ECp5jexhWXVqkjV840Vw_F-Gykt_VMfd.PDF
-
.\data\BATCH2.BCH
SAMPCO
LB1
1234567
T1
COMPANY COPY
DATA\0v3l7pBdVqHceoRL5hf2xqjJ7WAMxiRVO9U70iFiXIcne.PDF
```

You can use the INI options in the DocSetNames control group to determine which XML elements are created. The values are the same as those written to a recipient batch or transaction file.

The MultiFilePrint callback function should only be used with the PDF, RTF, HTML, and XML print drivers.

MAPPING FIELDS WITH XPath

The GenTrn program and the NoGenTrnTransactionProc rule let you use the TRN_Fields control group to map all of your fields with XPath. To let the system know you are using the XML file, set the XMLTrnFields option in the TRN_File control group to Yes and also set the XMLExtract option in the RunMode control group to Yes.

Here is an example:

```
< RunMode >
    XMLExtract = Yes
< TRN_File >
    XMLTrnFields= Yes
< TRN_Fields >
    Company    = !/Forms/Key1
    LOB        = !/Forms/Key2
    PolicyNum  = !/Forms/PolicyNum
    RunDate    = !/Forms/RunDate;DM-4;D4
```

NOTE Use this format for the Trn_Fields control group options:

(Field in the Transaction DFD File XPath;Field Format

Be sure to include the leading exclamation mark (!). This tells the system to use an XML path search but is not part of the actual search routine. Do not specify whether a field is a key. The system does not support multiple (search) keys with the XML implementation.

If you are selectively excluding transactions, in your exclude file, instead of an offset and SearchMask, replace it with the XPath. Here is an example:

```
!/Forms[PolicyType="OLD"]
```

RUNNING ARCHIVE IN SINGLE STEP PROCESSING

Using rules developed for archiving via IDS, you can run the GenArc program as part of single-step processing.

Use the InitArchive rule to check the INI options in the Trigger2Archive control group, initialize the database, open the APPIDX.DFD and CAR files, and perform other steps to initialize archive.

The Archive rule then unloads the current form set and converts field data for archive using the INI options in the Trigger2Archive control group.

Here is an example:

```
< Base Rules >
;InitArchive;1;;
< Base Form Set Rules >
;Archive;2;;
```

NOTE For more information on these rules, see the [Rules Reference](#)

RUNNING WIP IN SINGLE STEP PROCESSING

You can use the InitConvertWIP and ConvertWIP rules to run the GenWIP program in single-step mode.

Use the InitConvertWIP rule to perform the initialization necessary for the ConvertWIP rule.

Use the ConvertWIP rule to see if the current transaction is assigned to the MANUAL.BCH file. If it is, the rule adds the record to WIP and unloads the contents of the POLFILE.DAT and NAFILE.DAT files to new files with unique names.

You can then view these WIP records using Documaker Workstation or Print Preview, which is part of the Internet Document Server (IDS).

Here is an example:

```
< Base Rules >
;InitConvertWIP;1;;
< Base Form Set Rules >
;ConvertWIP;2;;
```

NOTE For more information on these rules, see the [Rules Reference](#)

RULES USED IN SINGLE STEP PROCESSING

Specific rules are used to combine the execution and functionality of the GenTrn, GenData, and GenPrint programs into a single step. To begin familiarizing yourself with these rules, an alphabetical listing and brief description follows. You can find more information in the Rules Reference.

Archive Use this form set level (level 2) rule after the InitArchive rule to unload the current form set and convert field data for archive using the INI options in the Trigger2Archive control group.

BatchingByRecipINI Use this form set level (level 2) rule to send transactions to a batch you specify based on data in the extract file. To use this rule, you must include the BatchingByRecip control group in your FSISYS.INI file with options similar to those shown below:

```
< BatchingByRecip >
  Batch_Recip_Def = default;"ERROR"
  Batch_Recip_Def = 4,1234567;"BATCH1";INSURED
  Batch_Recip_Def = true;"BATCH2";INSURED
  Batch_Recip_Def = True;"BATCH3";COMPANY | true;"BATCH2";AGENT
```

You must also add the TWOUP control group and CounterTbl option to the FSISYS.INI file.

BatchByPageCount Use this form set level rule to send a transaction to a specific batch based on the number of pages produced by processing the transaction. The batch used is determined by the PageRange option in the Batch control group.

In the example below; transactions that produce 1 to 7 pages are send to Batch1. Transactions that produce 8 to 25 pages are send to Batch2. In addition, you must add the TWOUP control group and CounterTbl option to the FSISYS.INI file.

```
< Batches >
  Batch1      = .\data\Batch1
  Batch2      = .\data\Batch2
  Batch3      = .\data\Batch3
  Error       = .\data\Error
  Manual      = .\data\Manual

< Batch1 >
  Printer     = Batch1_PTR
  ...        .....
  PageRange  = 1,7

< Batch2 >
  Printer     = Batch2_PTR
  ...        .....
  PageRange  = 8,25
  .....

< TWOUP >
  CounterTbl = .\data\counter.tbl
```

BuildMasterFormList Use this job level rule (level 1) to load the FORM.DAT file into an internal linked list within the GenData program. You must include this rule in the AFGJOB.JDT file because the RunSetRcpTbl rule is dependent on the list this rule creates.

ConvertWIP Use this form set level (level 2) rule to see if the current transaction is assigned to the MANUAL.BCH file. If it is, the rule adds the record to WIP and unloads the contents of the POLFILE.DAT and NAFILE.DAT files to new files with unique names. You can then view these WIP records using Documaker Workstation or Print Preview, which is part of the Internet Document Server (IDS).

InitArchive	Use this job level (level 1) to check the INI options in the Trigger2Archive control group, initialize the database, open the APPIDX.DFD and CAR files, and perform other steps to initialize archive.
InitConvertWIP	Use this job level (level 1) rule to perform the initialization necessary for the ConvertWIP rule.
InitPrint	Use this job level (level 1) rule to load printer and recipient batch information. This rule sets up PRTLIB data, initializes print options, and loads a table which contains page totals for recipient batch files.
InitSetRecipCache	Use this job level rule (level 1) to set the amount of cache the system uses to store recipient information in memory. With this rule you can tell the system the amount of memory to set aside and use for storing information in the Key1, Key2, and Key3 fields, often used to store the company, line of business, and transaction codes. You can use this rule to improve processing performance for complex forms. This rule has no effect on the processing speed for static forms.
NOTE If you omit this rule, the system does not set aside memory for the Key1, Key2, and Key3 fields.	
NoGenTrnTransactionProc	Use this form set level rule when you use the GenData program by itself to execute the GenTrn and GenData steps. In the single-step processing environment, this rule processes the extract file and creates the information normally created in both the GenTrn and GenData steps. When combined with the InitPrint and PrintFormset rules, it creates the output files normally created during the GenPrint step.
NOTE Do not use this rule if you are running the GenTrn, GenData, and GenPrint programs as separate processes (multi-step processing).	
PageBatchStage1InitTerm	Use this job level rule (level 1) to create and populate a list of records which contain page ranges and total page counts for each recipient batch file. This rule is typically used for handling 2-up printing for AFP and compatible printers. This rule creates a list (populated in another rule) to contain the recipient batch records for a multi-mail transaction set. The rule then writes out the recipient records for the final multi-mail transaction set and writes out the total page counts for each recipient batch. You must add the TWOUP control group and CounterTbl option to the FSISYS.INI file, as shown here: <pre>< TwoUp > CounterTbl = .\data\counter.tbl</pre>
PaginateAndPropagate	Use this form set level (level 2) rule to paginate the form set and merge in or propagate field data.
PrintFormset	Use this form set level (level 2) rule when you run the GenData program by itself to execute GenTrn and GenPrint processes. In the single-step processing environment, this rule, when combined with the InitPrint rule, prints form sets.

NOTE Do not use this rule if you are running the GenTrn, GenData, and GenPrint programs as separate processes (multi-step processing).

- ProcessQueue** Use this form set level (level 2) rule to process the queue you specify. This rule loops through the list of functions for the queue you specify and then frees the queue when finished.
- StandardFieldProc** This rule is a field level rule (level 4), which you must include in the AFGJOB.JDT file. This rule is used when you are using the performance mode JDT and should be the first field level rule. This rule tells the system to process each field on all of the images triggered by the SETRCPTB.DAT file. If you use the StandardFieldProc rule in your JDT, you must also include the WriteNAFile rule.
- StandardImageProc** This rule is an image level rule (level 3) which you must include in the AFGJOB.JDT file. This rule is used when you are using the performance mode JDT and should be the first image level rule. This rule tells the system to process each image triggered by the SETRCPTB.DAT file.
- WriteNAFile** Use this form set level rule (level 2) to append the NAFILE.DAT file data records for the current form set into an existing NAFILE.DAT file. When you use the NoGenTrnTransactionProc rule, which replaces the RULStandardProc rule, you must include the WriteNAFile rule to cause data (records) to be written to the NAFILE during the GenData processing step. In addition, you must also include the WriteOutput rule to cause data (records) to be written to the POLFILE.DAT and NEWTRN.DAT files during the GenData processing step.
- WriteOutput** Use this form set level (level 2) rule to append the POLFILE.DAT file data records for the current form set into an existing POLFILE.DAT file.
- You also use this rule when you are using the GenData program by itself to execute the GenTrn, GenData, and GenPrint processing steps.
- If you use this rule, do not use the UpdatePOLFile rule.
- WriteRCBWithPageCount** Use this form set level rule (level 2) to write page counts for each recipient. This rule is typically used for handling 2-up printing on AFP and compatible printers. To use this rule, you must update the RCBDFDFL.DFD file with the following items:

```
< FIELDS >
  FIELDNAME = CurPage
  FIELDNAME = TotPage
  FIELDNAME = AccumPage
  FIELDNAME = MMFIELD
< FIELD:CurPage >
  INT_TYPE = LONG
  EXT_TYPE = CHAR_ARRAY_NO_NULL_TERM
  EXT_LENGTH = 10
  KEY = N
  REQUIRED = N
< FIELD:TotPage >
  INT_TYPE = LONG
  EXT_TYPE = CHAR_ARRAY_NO_NULL_TERM
  EXT_LENGTH = 10
```

```

        KEY = N
        REQUIRED = N
    < FIELD:AccumPage >
        INT_TYPE = LONG
        EXT_TYPE = CHAR_ARRAY_NO_NULL_TERM
        EXT_LENGTH = 10
        KEY = N
        REQUIRED = N
    < FIELD:MMFIELD >
        INT_TYPE = CHAR_ARRAY
        INT_LENGTH = 7
        EXT_TYPE = CHAR_ARRAY_NO_NULL_TERM
        EXT_LENGTH = 6
        KEY = N
        REQUIRED = N

```

SINGLESTEPPROCESSINGEXAMPLE

As stated earlier, the single-step process is performed by combining the execution and functionality of the GenTrn, GenData, and GenPrint programs. This is done by placing certain rules into a specialized JDT. The earlier illustration shows the input and output files used by GenData to process transactions and print output files in one step. The following file describes the JDT used to process the job and an example of the rules used to combine the GenTrn, GenData, and GenPrint functions.

To make this happen, the NoGenTrnTransactionProc rule, along with other rules, are placed in the JDT file as seen in the following sample file. An actual sample file can be seen in the RPEX1 sample library.

Base rules

The following base rules are designed for the performance mode.

```

;RULStandardJobProc;1;Always the first job level rule;
;SetErrHdr;1;***:-----;
;SetErrHdr;1;***: BillPrint Data Generation (Base);
;SetErrHdr;1;***:;
;SetErrHdr;1;***: Transaction:      ***ACCOUNTNUM***;
;SetErrHdr;1;***: Company Name:    ***Company***;
;SetErrHdr;1;***: Line of Business: ***LOB***;
;SetErrHdr;1;***: Run Date:       ***RunDate***;
;SetErrHdr;1;***:-----;
;JobInit1;;
;CreateGlbVar;1;TXTLst,PVOID;
;CreateGlbVar;1;TblLstH,PVOID;
;InitOvFlw;1;;
;SetOvFlwSym;1;SUBGROUPOVF,SUBGROUP,5;
;BuildMasterFormList;;4;
;PageBatchStage1InitTerm;;;
;InitSetrecipCache;;;

```

The following rule is required to execute GenData and GenPrint as a single step.

```

;InitPrint;;;

```

Base form set rules The following base form set rules causes GenTrn and GenData to be combined into a single step.

```
;NoGenTrnTransactionProc;;;
;ResetOvFlw;2;;
;BuildFormList;;;
;LoadRcpTbl;;;
;RunSetRcpTbl;;;
```

The following rules are required to execute GenData and GenPrint as a single step.

```
;PrintFormset;;;
;WriteOutput;;;
;WriteNaFile;;;
;WriteRCBWithPageCount;;;
;ProcessQueue;;PostPaginationQueue;
;PaginateAndPropagate;;;
;BatchingByRecipINI;;;
```

Base image rules The following base image rules apply to every image in this base.

```
;StandardImageProc;3;Always the 1st image level rule;
```

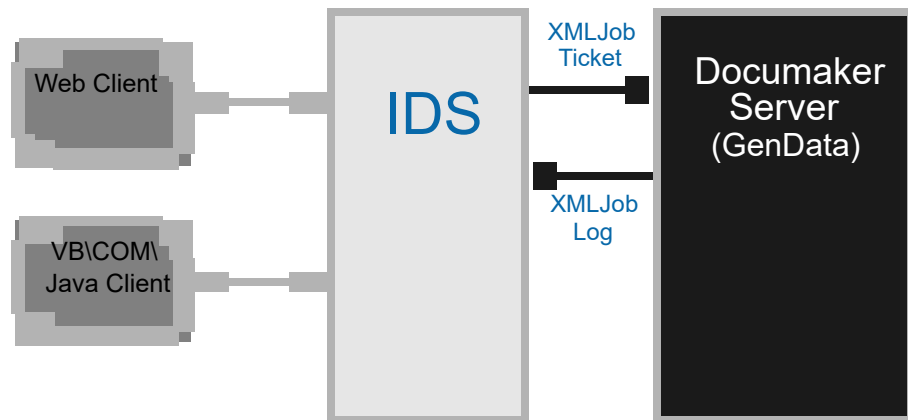
Base field rules The following base field rules apply to every field in this base.

```
;StandardFieldProc;4;Always the 1st field level rule;
```

USING IDS TO RUN DOCUMAKER

If you have a license for both Documaker and the Internet Document Server (IDS), you can set up IDS to run Documaker as a subordinate process. Web clients communicate with IDS using queues. IDS communicates with Documaker via XML files called *tickets* and *job logs*.

This diagram illustrates the process:



IDS can start or stop Documaker Server as needed, without user interaction. One IDS session controls one Documaker process. You can, however, implement multiple IDS sessions and have multiple Documaker Server processes as well.

Keep in mind these limitations:

- You can only run Documaker in single step mode.
- You must run Documaker on Windows 2000 or higher.
- Different resource setups for Documaker are supported, but Documaker processing restarts if resources are changed, eliminating the performance benefits. This should not be a problem because it is unlikely multiple Documaker Server setups will be used with a single IDS implementation. You can, however, experience problems testing a system with multiple setups.
- During processing, some INI options can be changed by the client. Since some Documaker rules use static variables and store INI values in memory, it is possible that a client will be unable to change an INI option if those Documaker rules are used. To handle these situations, you must restart Documaker.

For more information, see the [Internet Document Server Guide](#) and the [SDK Reference](#).

WRITING UNIQUE DATA INTO RECIPIENT BATCH RECORDS

The GenData program lets you add unique data to each recipient batch record before it is written to the recipient batch files. The recipient batch record data and format is defined by the GVM variable definitions in the RCBDFDFL.DAT file.

You can use this capability if you need to add...

- Address information or other field level information to the batch record, which is typically unique for each recipient.
- Recipient information that is not handled by normal field mapping from the transaction DFD to the recipient batch DFD.
- Cumulative or calculated information not available until the document is nearly completed.

Understanding the System

Before this feature was implemented in version 10.2, the recipient batch records were identical except for the recipient code field which contains a unique identifier assigned to a given recipient. If additional recipient data was required, you had to write a custom rule.

Use the options in the RecipMap2GVM control group to set up this capability. Data that can be added to the recipient batch record can be:

- Contents of a variable field on the specified image or form/image
- Constant value
- Data from an existing INI built-in functions, such as ~DALRun
- Data from a custom written INI function

Here is an example of the RecipMap2GVM control group:

```
< RecipMap2GVM >
  Form      =
  Image     =
  Req       =
  Opt       =
```

Option Description

Form	(optional) Enter the name of the form.
Image	Enter the name of the image. You can also enter an image name root. An image name root is the first part of a name. For instance, MAILERS is the root name for images with names such as MAILER_1 , MAILER_2 , or MAILERS .
Req *	A semicolon delimited string that contains one of the following: - GVM variable name; variable field name; optional formatting information - GVM variable name; blank character (space); constant value - GVM variable name; INI built-in function

Option Description

Opt * A semicolon delimited string that contains one of the following:

- GVM variable name; variable field name; optional formatting information
- GVM variable name; blank character (space); constant value
- GVM variable name; INI built-in function

* = Repeat for each GVM variable you are setting up.

Optional formatting information

You can add optional formatting information as a parameter of the Opt INI option. This formatting information is comprised of four items separated by commas.

Item	Description
Input fetypes	D or d = date N or n = number
Input format mask	Date - see the FmtDate rule in the Rules Reference. Number – see the FmtNum rule in the Rules Reference.
Output fetypes	D or d = date N or n = number
Output format mask	Date - see the FmtDate rule in the Rules Reference. Number – see the FmtNum rule in the Rules Reference.

Here are some formatting examples:

```
d, "1/4", d, "4/4"
```

This converts an input date, mmddyyyy, into *month name dd, yyyy* such as February 17, 2006.

```
n, nCAD, nUSD, "$zzz,zz9.99"
```

This converts an input numeric value in Canadian French format into a value in United States format.

Keep in mind...

- For the Req option, if the data is missing an error occurs and the transaction is send to the error batch.
- For the Opt option, if the data is missing the system stores an empty string in the GVM variable.
- A RCB GVM variable cannot be restored to its original or default value after it has been changed using this method.
- Any RCB GVM variable not assigned using this method retains the value originally set during the transaction processing.
- Some RCB GVM variables should never be changed using this mapping technique. These include:

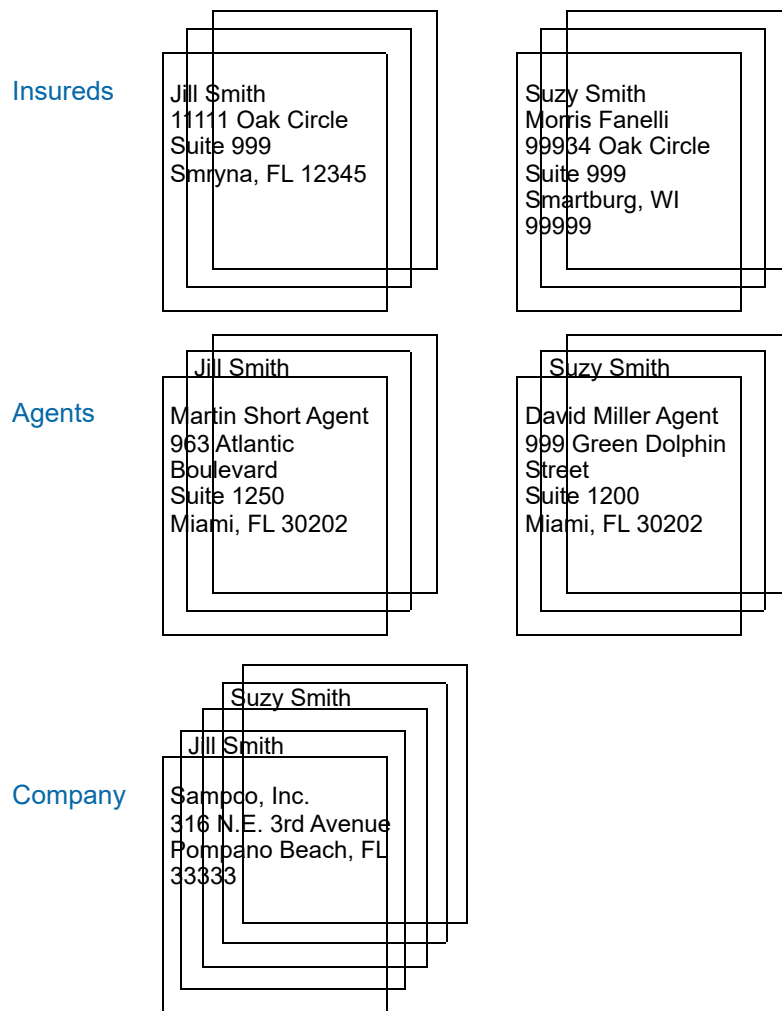
TRN_Offset

NA_Offset

POL_Offset

- If the image defined in the Image option in the RecipMap2GVM control group does not name an image, the feature is disabled for all transactions.
- If the image defined in the Image option is missing from the form set being processed, the GVM data is not changed. Depending on where the GVM data is mapped, this could mean data from the prior transaction will still be in the GVM variables.
- If there are multiple images with the same name in the form set, the form specified in the Form option is used to identify the image to use. If the Form option is omitted, the first image found in the current form set is used.
- The system assumes the specified image contains all of the unique data except for a constant value or data gathered from an INI built-in function.
- If more than one recipient is assigned to the image, all recipient batch records receive the same added data.

Example This example creates a mailer cover page for each insured, agent, and/or company recipient per transaction. The cover page is created using banner page processing which occurs during GenPrint processing. Examples of the three different mailer cover pages are as follows.



This example assumes that the:

- Agent and company recipient batch files are sorted (agent number and company name, respectively) before the GenPrint program runs. This sorting allows for the creation of only one mailer cover page per unique agent and company.
- Unique information is contained on the form/image, Dec Page/Q1MDC1.
- The FSIUSER.INI file includes these control groups and options:

```
< RecipMap2GVM >
  Form      = Dec Page
  Image     = Q1MDC1
  Opt       = Name1;Insured Name;
  Opt       = Name2;Insured Name2;
  Opt       = Address1;Address Line1;
  Opt       = Address2;Address Line2;
  Opt       = CityCounty;prtvalue;
  Opt       = AgentName;Agent Name;
  Opt       = AgentID; Agent ID;
  Opt       = OfficeAddress;Office Address;
  Opt       = TownandState;Town And State;
< Printer >
  PrtType           = PCL
  EnableTransBanner = True
  EnableBatchBanner = False
  TransBannerBeginScript= PreTrans
  TransBannerEndScript = PstTrans
  TransBannerBeginForm = ;BANNER;TRANSACTION;TRANS HEADER;
  TransBannerEndForm   = ;BANNER;TRANSACTION;TRANS TRAILER;
< DALLibraries >
  LIB = Banner
```

BANNER.DAL The DefLib directory contains this DAL script:

```
* This script obtains the required unique data from the recipient
* batch record and stores it on the mailer form.
```

```
BeginSub PreTrans

blank_gvm = Pad(" ",41," ")
SetGVM("NameA"      ,blank_gvm,, "C",41)
SetGVM("NameB"      ,blank_gvm,, "C",41)
SetGVM("AddressA"    ,blank_gvm,, "C",41)
SetGVM("AddressB"    ,blank_gvm,, "C",41)
SetGVM("CityCounty1" ,blank_gvm,, "C",41)
If Trim(RecipName()) = "INSURED" Then
  SetGVM("NameA"      ,GVM("Name1")      ,,"C",41)
  SetGVM("NameB"      ,GVM("Name2")      ,,"C",41)
  SetGVM("AddressA"    ,GVM("Address1")   ,,"C",41)
  SetGVM("AddressB"    ,GVM("Address2")   ,,"C",41)
  SetGVM("CityCounty1" ,GVM("CityCounty") ,,"C",41)
  GoTo exit:
End

last_agent_id = last_agent_id
If Trim(RecipName()) = "AGENT" Then
```

```

If last_agent_id != Trim(GVM("AgentID")) Then
  last_agent_id = Trim(GVM("AgentID"))
  SetGVM("NameA"      ,GVM("AgentName")      ,,"C",41)
  SetGVM("NameB"      ,GVM("OfficeAddress")  ,,"C",41)
  SetGVM("AddressA"    ,GVM("TownandState")  ,,"C",41)
  GoTo exit:
Else
  SuppressBanner()
  GoTo exit :
End
End

last_company_name = last_company_name
If Trim(RecipName()) = "COMPANY" Then
  If Trim(GVM("Company")) != last_company_name Then
    last_company_name = Trim(GVM("Company"))
    If Trim(GVM("Company")) = "SAMPCO" Then;
      SetGVM("NameA"      ,"Sampco, Inc."      ,,"C",41)
      SetGVM("NameB"      ,"316 N.E. 3rd Avenue" ,,"C",41)
      SetGVM("AddressA"   ,"Pompano Beach, FL 33333" ,,"C",41)
      GoTo exit:
    ElseIf Trim(GVM("Company")) = "FSI"
      SetGVM("NameA"      ,"FSI Inc."          ,,"C",41)
      SetGVM("NameB"      ,"222 Newbury St."    ,,"C",41)
      SetGVM("AddressA"   ,"Northwest City, FL 99999" ,,"C",41)
      GoTo exit:
    End
  Else
    SuppressBanner()
    GoTo exit:
  End
End
exit:
EndSub

BeginSub PstTrans
EndSub

```

The RCBDFFDL.DAT file contains the following GVM variable definitions which are defined in the RecipMap2GVM control group:

- Name1
- Name2
- Address1
- Address2
- CityCounty
- AgentName
- AgentID
- OfficeAddress
- TownAndState

Here are two recipient batch records from this example:

```
SAMPCOLB12234567SCOM1FLT1 B2199802232234567890      0 22560
*****001      3724      452Jill Smith      Morris
11111 Oak Circle      Suite 999      Smyrna,
FL 12345      Martin Short Agent      963 Main Street,
Suite 1250 Miami, FL 30202

FSI CPP4234567FSIM1WIT1 B3199802234234567890      0 30360
*****001      4667      565Suzy Smith      Morris
99934 Oak Circle      Suite 999      SmartBurg,
WI 99999      David Miller Agent      999 Main Street,
Suite 1200 Miami, FL 30202
```

USING CLASS RECIPIENTS

A *class recipient* identifies a recipient that represents one or more persons or entities. For instance, in an insurance implementation, you might have a policy that has a several recipients declared as *Additional Interest*. Instead of declaring each as a separate recipient with separate triggering logic, it is more convenient to declare a single recipient name that represents all those of the same type or class. All members of this class receive virtually identical copies of the document.

In this scenario, you do not have to do anything special to declare a class recipient in your form definitions. Merely determine the appropriate title for this class of recipients and define that name as you would a normal recipient that represents a single entity.

If you want all members of the class to receive identical copies of the document, use the trigger for the recipient to assign a copy count to each form or image — where the count equals the number of members in the class.

There are some limitations to using form copy counts to provide recipient copies. For instance, this does not let you print unique information about each member of the class recipient, as would be necessary on a mailer page, for instance.

NOTE It is possible to handle this using trigger overflow processing to physically trigger multiple copies of each form — one for each member, but a disadvantage of this approach is that each item (form or image) triggered is physically duplicated in the form set and therefore each requires data processing. This means that if there are a large number of these duplicate recipients, the throughput performance of transactions could be affected.

To handle this situation, the RecipMap2GVM feature can write additional batch records for each member of a class recipient. The RecipMap2GVM feature lets you write unique recipient information to each batch record.

With this method, only a minimal amount of additional processing occurs in the form set mapping. Yet, because a separate batch record is written for each member, the system prints a separate copy of the document for each member and you can use the unique information saved in each batch record to provide a unique banner page, such as a mailer, for each member in the print output.

To use the RecipMap2GVM feature, follow these steps:

- 1 Add an image to your form set definition and assign this image the name of your class recipient. Normally, you would also flag this image ~~hidden~~, since you would not want it to display or print. This purpose of this image is to hold the unique information for each member of the class recipient.
- 2 Define a trigger for the image that uses overflow to generate as many copies of the image as there are members in the data. The idea is to trigger an instance of the image for each member recipient. Be sure to also declare and create the appropriate overflow variable in the AFGJOB.JDT file you will use during data mapping.
- 3 Create the image and add fields that map the data to be written to the batch record for each member. Be sure to use the appropriate overflow variable for this image in your rule mapping definitions. Also remember to assign the appropriate image level rule to increment the overflow symbol after processing each image.

- 4 Set up your RecipMap2GVM INI control group and modify your RCBDFDFL.DFD (Recipient Table DFD) file to include your unique data fields for the recipient batch records. Specify the new image as the image required in the RecipMap2GVM control group and set up each of the fields to map into your RCBDFDFL.DFD file layout.

NOTE See [Writing Unique Data into Recipient Batch Records on page 48](#) for more information on the RecipMap2GVM control group.

When you run the GenData program, your new image will trigger once for each member recipient. During normal processing, the fields on each image will map (using overflow variables) the unique data for each member. Because you have multiple copies of the image triggered, the RecipMap2GVM feature creates a separate batch record for each instance of the image. Therefore, you receive a separate record representing each individual member of your class recipient.

When the GenPrint program runs, having a separate record for each class recipient in the batch causes that transaction to print once for each member. And by using banner page processing, you can take the unique information written into each batch record and map that information to a mailer page, making the final output unique to each member of the class.

RUNNING DOCUMAKER USING XML JOB TICKETS

You can run Documaker from another application using an XML job ticket. You receive results in an XML job log file.

The layout of these files is the same as those used by IDS for running Documaker. See [Using IDS to Run Documaker on page 67](#) for more information.

The name of the job ticket is passed to the GenData program on the command line as

```
/jticket= parameter
```

The default name is **JOBTICKET.XML**

To set this up replace the StandardJobProc rule with the TicketJobProc rule. Keep in mind you must run Documaker in single step mode, since only the GenData program is executed. See [Single-step Processing on page 36](#) for more information.

You can specify the name of the resulting job log file using this command line parameter:

```
/jlog=
```

The default is **JOBLOG.XML**

HANDLING 2-UP PRINTING

Two-up printing lets you print two transactions on the same page of single- and multi-page forms. 2-up printing is a two-step process which passes input through GenData three (3) times, using a different JDT file each time.

This process is similar to the single-step process in that GenData performs the work, but the three passes through GenData actually represent two steps of the multi-step process: processing the transactions and printing the transactions.

For more information and to see example JDT files, see [Single-step Processing Example on page 45](#)

NOTE 2-up printing is only available for AFP printers.

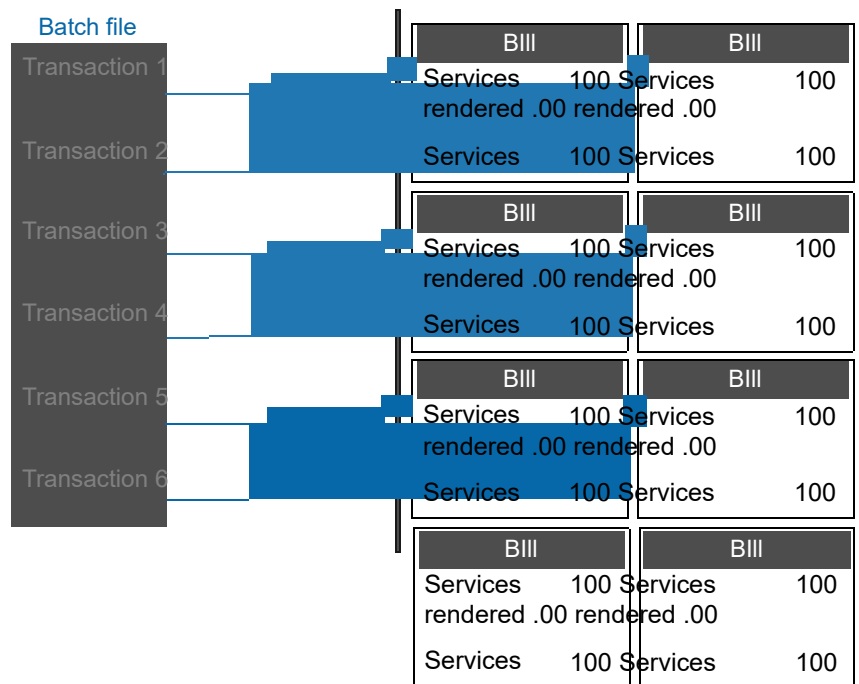
There are several scenarios in which 2-up printing applies:

- 2-up printing with single-page forms
- 2-up printing with multi-page forms

The following illustrations describe these scenarios.

2-up printing with single-page forms

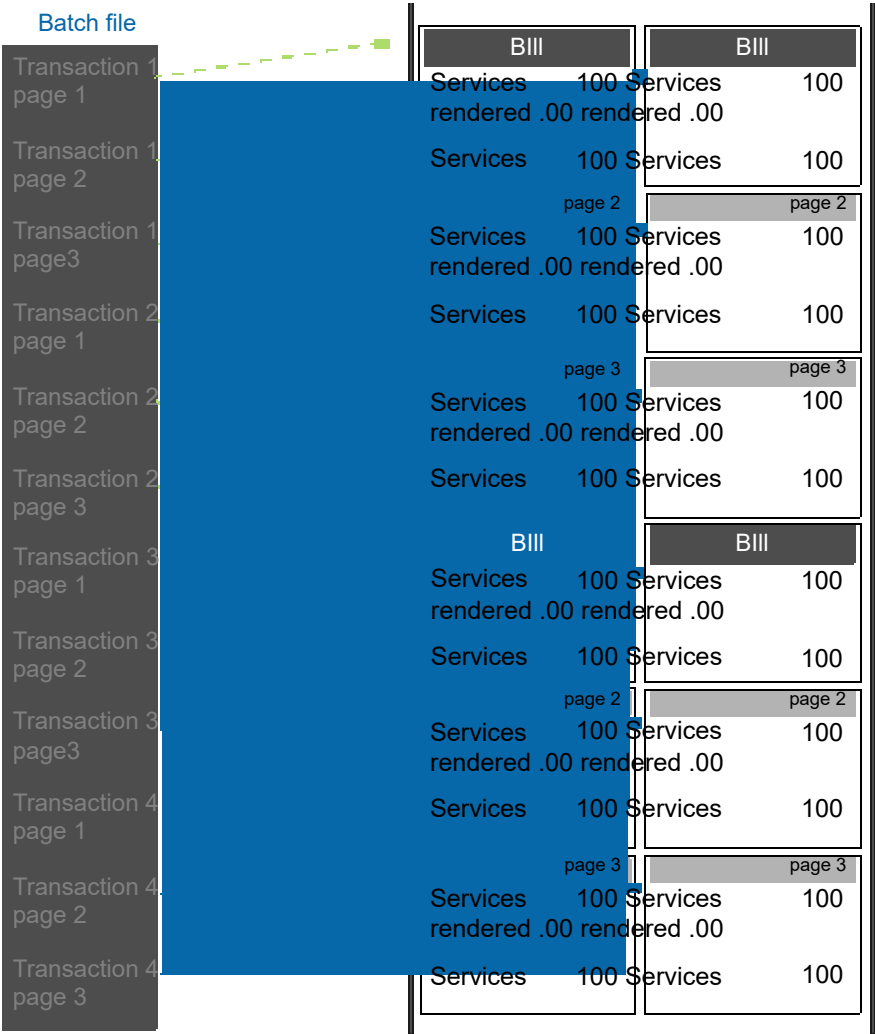
This illustration shows how 2-up printing works when you use single-page forms, such as some types of bills and statements.



In this scenario, the system merges the data for the first transaction onto the form and then prints the form.

2-up printing with multi-page forms

This illustration shows how 2-up printing works when you use multi-page forms.



Changing the INI File

You must make the following changes in your FSISYS.INI file.

NOTE Changes to the error and manual recipient batch control groups are not necessary.

- You must include a Printer option in the recipient batch control groups for each print file created. These printers must also be defined in the FSISYS.INI file.
- The recipient batch groups must have a FinalPrinter option. This option specifies the printer to use for the final, merged file. This printer must also be defined in the FSISYS.INI file.

- The recipient batch groups must have a PageRange option for page count batching. You specify this option as shown below:

```
PageRange = [min], [max]
```

If you do not specify *min*, the system uses zero (0). If you omit *max*, the system uses (unsigned) -1 (all bits on). The *min* and *max* values are inclusive.

- You can also include in the recipient batch control groups a TwoUpStart option, which can have any of these values (case is irrelevant):

L
Left
R
Right

This option specifies whether the merge process should associate the first Printer option with the left or the right side of the page. The system only checks this option when there are multiple Printer options present in the control group. If you omit this option, the file specified in the first Printer option is used for the left side of the page.

Here is an example of a recipient batch control group:

```
< Batch1 >  
Printer      = Printer1  
Printer      = Printer2  
FinalPrinter = Printer3  
PageRange = ,1  
TwoUpStart = R
```

This splits single page transactions evenly between the files specified in the Printer1 and Printer2 control groups. The files specified in the Printer1 and Printer2 control groups will then be merged into the file specified in the Printer3 control group. The file specified in the Printer1 control group is used for the right page.

Creating the TWOUP control group

You must create the TwoUp control group. This control group must contain the CounterTbl option, which specifies the file name for the table that contains recipient batch page counts.

The TwoUp control group can optionally contain the CounterDFD option, which specifies the name of a DFD file. See the [Rules Reference](#) for information about this DFD.

The TwoUp control group can optionally contain the LMargin, LShift, and RShift options. Records on the left page will be shifted to the right by LShift - LMargin, and records on the right page will be shifted to the right by RShift - RMargin. Amounts are in FAP units (2400 per inch). If you omit these options, the system uses these defaults:

```
LMargin = 600  
LShift = 1200  
RShift = 16800  
  
< TwoUp >  
CounterTbl = data\counter.tbl  
CounterDFD = deflib\counter.dfd  
LMargin = 300
```

```
LShift = 600
RShift = 15000
```

The first two options define the location of the files shown above.

The LMargin=300 option sets the left margin to 1/4 inch. The LShift=600 option shifts the left page 1/2 inch from the left edge of the paper (1/4 inch beyond the left margin). The RShift=15000 option shifts the right page 6 1/2 inches the left edge of the paper (6 inches from the left margin).

Creating the Added_Fonts control group

You can optionally create the Added_Fonts control group. The options in this group specify additional fonts to add to the AFP output file for text label records which may be added during the merge process. Each option takes the form:

```
FontName = <font name>
```

Here is an example:

```
< Added_Fonts >
  FontName = XOFATINO
  FontName = XOFAUNN8
```

This tells the system to include the fonts XOFATINO and XOFAUNN8 in the final output file, regardless of whether they are present in the input files.

Changing the Recipient Batch DFD File

The recipient batch DFD file (RCBDFDFL.DAT) must have the following fields with the given types. You can modify the field lengths—just make sure you set the EXT_LENGTH option large enough to represent all of the pages in a multi-mail transaction set. Also make sure you set the INT_LENGTH option larger by one than the EXT_LENGTH option.

Note that the field name is case sensitive. Also, for each of these fields, be sure to add a FIELDNAME= line to the <FIELDS> line in the DFD file.

```
< FIELD:CurPage >
  INT_Type = CHAR_ARRAY
  INT_Length = 5
  EXT_Type = CHAR_ARRAY_NO_NULL_TERM
  EXT_Length = 4
  Key = N
  Required = N
< FIELD:TotPage >
  INT_Type = CHAR_ARRAY
  INT_Length = 5
  EXT_Type = CHAR_ARRAY_NO_NULL_TERM
  EXT_Length = 4
  Key = N
  Required = N
< FIELD:AccumPage >
  INT_Type = LONG
  EXT_Type = CHAR_ARRAY_NO_NULL_TERM
  EXT_Length = 10
  Key = N
  Required = N
```

RULES USED FOR 2-UP PRINTING

The following descriptions will help familiarize you with the rules that are required to perform the 2-up printing process. All of the rules listed in the topic [Rules Used in Single-step Processing on page 41](#) are required for 2-up printing, plus these additional rules:

NOTE You can find more information in the [Rules Reference](#)

AddLine	Use this form set level (level 2) rule to add a line record, such as for OMR marks, to the AFP record list built by the MergeAFP rule.
AddTextLabel	Use this form set level (level 2) rule to add a text label record to the AFP record list built by the MergeAFP rule.
ForceNoImages	Use this image level rule (level 3) to return the msgNO_MORE_IMAGES message. This prevents errors if you have no image level rules.
GetRCBRec	Use this form set (level 2) level rule to set the current recipient batch file. This rule initializes the current recipient batch file, if necessary. This rule also sets the first printer for current batch to be the current printer and retrieves the next record from the current recipient batch file.
InitMerge	Use this job level (level 1) rule to create a list of printers, batches, and buffers for the comment (RCB) records. This rule also creates a list to hold AFP records and AFP fonts. After the system finishes running the rule, it deletes everything the rule created.
NOTE The recipient batch files are not used at this stage. The batch list must be created beforehand so the system will know which print files belong together. The <i>skipping batch</i> message is an artifact of the batch file loading process.	
InitPageBatchedJob	Use this job level (level 1) rule to open NA and POL files. This rule installs the image level callback function for inserting recipient batch records into the AFP print stream as AFP comment records. When finished, this rule restores the original callback function and closes the NA and POL files.

MergeAFP Use this form set level (level 2) rule to initialize input files. This rule populates the AFP record list, retrieves comment (RCB) records, and terminates the input files.

This rule also initializes output files, and writes out the AFP record list, adding end page and end document records as necessary. The rule then terminates these output files.

ParseCommentExample Use this form set level (level 2) rule to parse comment records into the GVM variable.

PrintData Use this form set (level 2) rule to print the form set. This rule is used for handling 2-up printing on AFP and compatible printers.

NOTE The image handler installed by the InitPageBatchedJob rule is called during the printing stage. If you want to make any modifications to the recipient batch record, you must do so before this point.

ProcessRecord Use this form set (level 2) rule to switch between print files as necessary when printing 2-up forms on an AFP printer. This rule updates the page count for current print file and loads and merges the form set.

Placing the 2-up Rules in the JDT File

When you use the rules listed at the beginning of this topic to handle 2-up printing, you must place them in the correct places and order in the AFGJOB.JDT file. Use the following table as a guide to where to place these rules. You can insert other rules before, between, or after the 2-up rules—just keep the 2-up rules in the order indicated below with respect to one another.

Stage 1		
	Job level	Insert the PageBatchStage1InitTerm rule after the RULStandardJobProc and JobInit1 rules
	Form set level	List the form set level rules in this order: WriteOutput CreateRecordList BatchByPageCount PaginateAndPropagate Place these rules after the RULStandardTransactionProc rule and make sure any rule which changes page count appears before these rules.
Stage 2		
	Job level	Include these rules in this order: InitPrint InitPageBatchedJob SetErrHdr <i>Do not</i> include the RULStandardJobProc or JobInit1 rules in this stage.
	Form set level	Include these rules in this order: GetRCBRec ProcessRecord PrintData <i>Do not</i> include the RULStandardTransactionProc rule in this stage.
	Image level	There are no regulations on the order in which you can place rules in this stage. Remember, however, that if there are no image level rules, you must include the ForceNoImages rule to avoid errors.
Stage 3		
	Job level	Place the InitMerge rule anywhere after the RULStandardJobProc rule.

Form set level	Make sure the MergeAFP rule is the first rule called. Place rules which add records or determine whether a page pair should be printed after the MergeAFP rule.
Image level	There are no stipulations on the order in which you must place rules in this stage. Remember, however, that if there are no image level rules, you <i>must</i> include the ForceNoImages rule to avoid errors.

2-UP PROCESSING EXAMPLE

As stated earlier, 2-up printing is a two-step process which calls GenData three times with different JDT files. These file excerpts show how to set up your batch and INI files:

2upbycnt.bat

You can set up this batch file as follows:

```
@Echo Off
SetLocal
Echo Y|Del Data\*. * >NUL
GenDaW32.Exe -INI=2upstep1.ini
If Not ErrorLevel 5 GoTo Step1NoError
    Echo "2Up Printing Failed in Step 1."
    GoTo Exit
:Step1NoError
GenDaW32.Exe -INI=2upstep2.ini
If Not ErrorLevel 5 GoTo Step2NoError
    Echo "2Up Printing Failed in Step 2."
    GoTo Exit
:Step2NoError
GenDaW32.Exe -INI=2upstep3.ini
If Not ErrorLevel 5 GoTo Step3NoError
    Echo "2Up Printing Failed in Step 3."
:Step3NoError
EndLocal
:Exit
```

2upstep1.ini

You can set up this INI file as follows:

```
< Data >
    AfgJobFile      = .\Def\AfgJob1.jdt
< Environment >
    FSISYSINI       = .\fsisys.ini
```

2upstep2.ini

You can set up this INI file as follows:

```
< Data >
    AfgJobFile      = .\Def\AfgJob2.jdt
< Environment >
    FSISYSINI       = .\fsisys.ini
```

2upstep3.ini

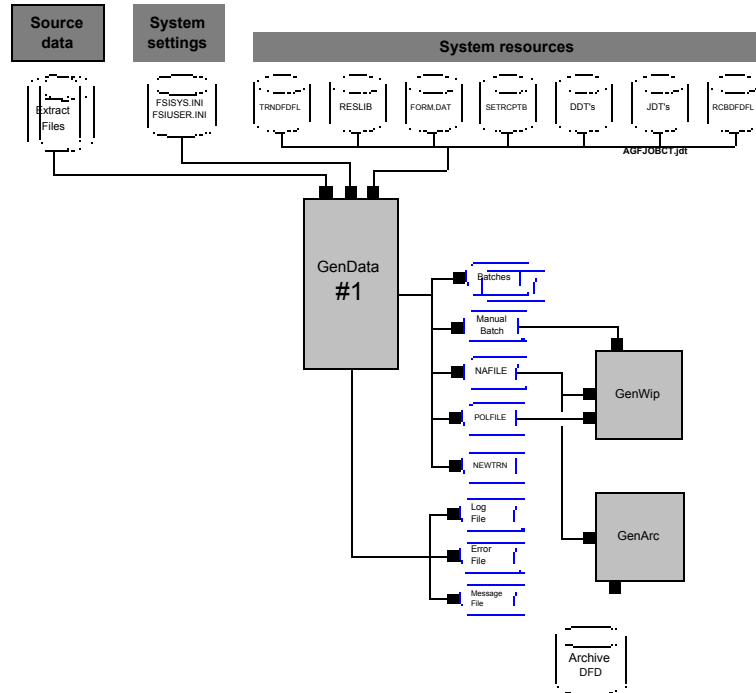
You can set up this INI file as follows:

```
< Data >
    AfgJobFile      = .\Def\AfgJob3.jdt
< Environment >
    FSISYSINI       = .\fsisys.ini
```

RUNNING THE GEN DATA PROGRAM

The following pages provide illustrations and an example files for each time the GenData program is run.

Step 1 - Using the AFGJOB1.JDT file



The first execution of GenData uses the AFGJOB1.JDT file with the base and form set rules shown in this example to create output files shown in the illustration.

```

<Base Rules>
;RULStandardJobProc;1;;
;SetErrHdr;1;***:-----;
;SetErrHdr;1;***: BillPrint Data Generation (Base);
;SetErrHdr;1;***:
;SetErrHdr;1;***: Transaction:      ***ACCOUNTNUM***;
;SetErrHdr;1;***: Company Name:    ***Company***;
;SetErrHdr;1;***: Line of Business: ***LOB***;
;SetErrHdr;1;***: Run Date:       ***RunDate***;
;SetErrHdr;1;***:-----;
;
;JobInit1;;;
;CreateGlbVar;1;TXTLst,PVOID;
;CreateGlbVar;1;TblLstH,PVOID;
;InitOvFlw;1;;
;SetOvFlwSym;1;SUBGROUPOVF,SUBGROUP,5;
;BuildMasterFormList;;4;
;PageBatchStage1InitTerm;;;
;InitSetrecipCache;;;

<Base Form Set Rules>
;NoGenTrnTransactionProc;;;
  
```

```

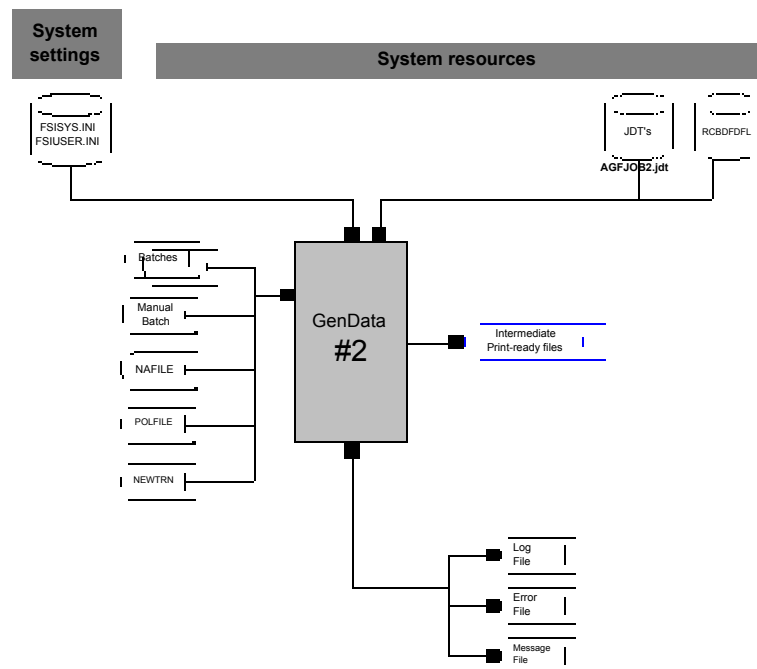
;ResetOvFlw;2;;
;BuildFormList;;;
;LoadRcpTbl;;;
;RunSetRcpTbl;;;
;WriteNaFile;;;
;WriteRCBWithPageCount;;;
;ProcessQueue;;PostPaginationQueue;
;WriteOutput;;;
;CreateRecordList;;
;BatchByPageCount;;
;PaginateAndPropagate;;;

<Base Image Rules>
;StandardImageProc;3;Always the 1st image level rule;

<Base Field Rules>
;StandardFieldProc;4;Always the 1st field level rule;

```

Step 2 - Using the AFGJOB2.JDT file



The second execution of GenData uses the AFGJOB2.JDT file. This JDT file uses the base and form set rules shown in this example to process the intermediate print files.

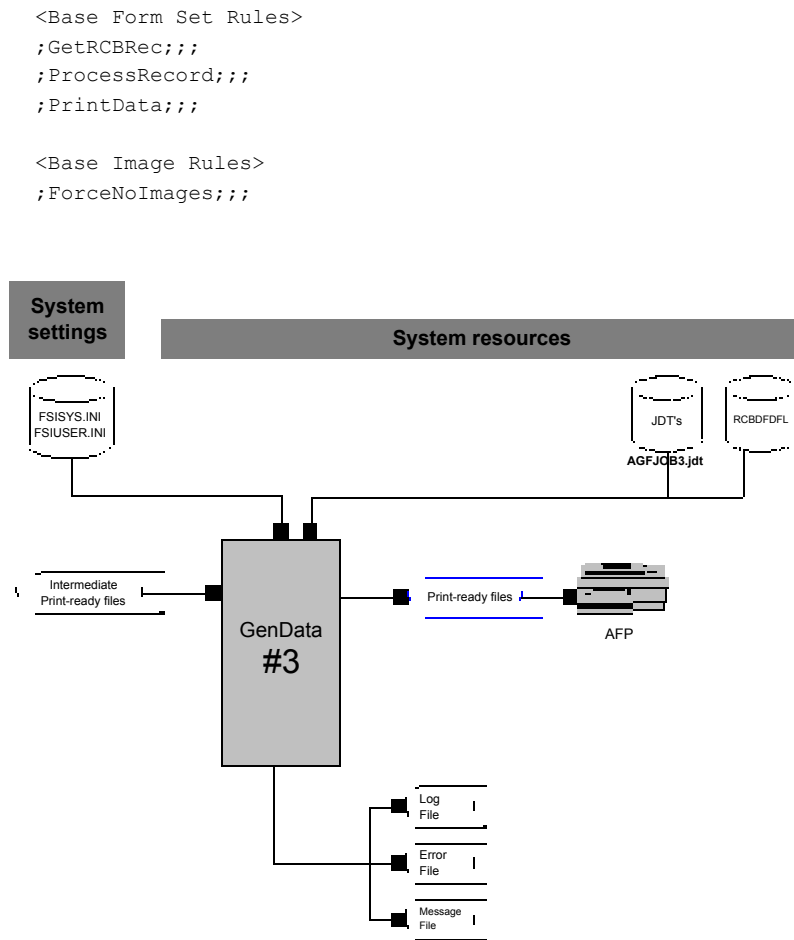
```

<Base Rules>
;InitPrint;;;
;InitPageBatchedJob;;;

;SetErrHdr;1;***:-----
;SetErrHdr;1;***: BillPrint Data Generation (Base) ;
;SetErrHdr;1;***: Company Name: ***Company***;
;SetErrHdr;1;***: SubCompany:***SubCompany***;
;SetErrHdr;1;***: Account #: ***AC-KY-BA***;
;SetErrHdr;1;***:-----

```

Step 3 - Using the
AFGJOB3.JDT file



The third execution of GenData uses the AFGJOB3.JDT file. This JDT file uses base and form set rules shown in this example to merge data intermediate print-ready files into a print-ready file for an AFP printer.

```

<Base Rules>
;RULStandardJobProc;;;
;SetErrHdr;1;***:-----;
;SetErrHdr;1;***: BillPrint Data Generation (Base);
;SetErrHdr;1;***: Company Name: ***Company***;
;SetErrHdr;1;***: SubCompany: ***SubCompany***;
;SetErrHdr;1;***: Account #: ***AC-KY-BA***;
;SetErrHdr;1;***:-----;
;InitMerge;;;

<Base Form Set Rules>
;MergeAFP;;;

<Base Image Rules>
;ForceNoImages;;;

```

SPLITTING RECIPIENT BATCH PRINT STREAMS

The GenPrint program and the PrintFormset rule (when running in single-step mode) are designed to produce one print stream output file for each recipient batch. This print stream output file includes all of the transactions in the recipient batch.

Sometimes, however, you may want to split the print stream output into multiple print stream output files. For instance, you can use this feature to split your batches into files that reflect the amount of paper you can load into your printer at one time.

You can use DAL scripts to set up criteria for splitting the output file to reflect almost any scenario. For example, it can be based on a certain number of transactions, a maximum number of sheets of paper, or on changes in variables in the recipient batch.

NOTE Some types of print streams require one file per transaction, such as RTF, PDF, and HTML. The typical way of handling this is via the multi-file print callback method, but this feature provides an alternate method which gives you greater control over the naming of the output file.

To do this you use the PrintFormset rule and these DAL functions:

- DeviceName
- SetDeviceName
- BreakBatch
- UniqueString

This rule and these DAL functions let you:

- Split recipient batches into multiple print stream files
- Assign names to those print stream files

For example, here are some things you can do:

Splitting batches by sheet count

You can use these functions to split a batch based on the sheet count during the GenPrint process. Once a batch reaches a certain number of sheets, you can tell the system to:

- Finish processing the current transaction
- End the current print file. (If you are using a post-transaction or post batch banner page, it will print before the file is closed.)
- Repeat this process when the next print file reaches the specified number of sheets

You can use virtually any logic to decide when to break the batch. For instance, to break based on sheet count, use the TotalSheets function to get the number of sheets to maintain a counter across the transactions.

NOTE Be sure to reset the sheet count variable in the pre-batch banner DAL script.

Here is an example of DAL script logic that might appear in a post-transaction banner DAL script:

```
IF TotalSheets() > 16000
  #COUNTER += 1
  CurFile = DeviceName()
  Drive = FileDrive(CurFile)
  Path = FilePath(CurFile)
  Ext = FileExt(CurFile)
  RecipBatch = RecipBatch()
  NewFile = FullFileName(Drive,Path,RecipBatch & #COUNTER,Ext)
  SetDeviceName(NewFile)
  BreakBatch()
END
```

NOTE See [Using DAL to Manipulate File Names on page 72](#) for information on using DAL functions to manipulate file names.

Creating PDF output

You can also modify the above script to unconditionally break the batch after each transaction. Assuming you used the SetDeviceName function to assign a proper file name, each recipient printed would receive a separate output file.

This is particularly useful for output types such as PDF, which require a separate file for each transaction.

NOTE You can also use the multi-file print callback method in GenPrint to get separate files. Similarly, the single-step processing mode currently uses this INI option:

```
< PrintFormset >
  MultiFilePrint = Yes
```

to tell the system to generate separate files for each transaction. Single-step mode automatically generates a unique file name and offers no way to override that name. By using the BreakBatch and SetDeviceName functions, however, you can control the names assigned to the files in single-step mode. To emulate the action of the current code, use the UniqueString function.

DAL functions

Here is a summary of the DAL functions you would use. Keep in mind...

- These print drivers are supported: PCL5, PCL6, PST, MET, AFP, PDF, HTML, and RTF.
- These print drivers *are not* supported: EPT, MDR, and GDI.
- All platforms are supported, but note that while UniqueString is supported on MVS, MVS does not support long file names.
- Producing PDF files on MVS requires a separate license. Contact your Skywire Software sales representative for more information.
- Both multi-step and single-step processing are supported.

The only DAL function actually involved in splitting the print stream is BreakBatch. The others make it easier to implement this functionality. For example, since you need to name the new print stream, you use the SetDeviceName procedure. To find the name of the current device, you use the DeviceName function. If you need to create unique file names, you can use the UniqueString function.

NOTE While you can call all of these DAL functions in the Rules Processor or Entry, the BreakBatch and SetDeviceName functions are not applicable in Entry since it does not use the batch printing engine. The other functions, DeviceName and UniqueString, are applicable to both Entry and the Rules Processor.

DeviceName

Use this function to return the current output device file name, such as the name of the current print stream output file.

Syntax DeviceName()

SetDeviceName

Use this procedure to set a new output device file name which will be used the next time the output device is opened, assuming nothing overrides the name prior to that.

Syntax SetDeviceName(Device)

BreakBatch

Use this procedure to tell the Rules Processor to break the output print stream file for the current recipient batch after processing the current recipient, including post transaction banner processing.

Syntax BreakBatch()

The procedure is typically called in the transaction banner DAL script. You must use the SetDeviceName function to specify a new device name. Otherwise, the new file has the same name as the old file and overwrites its contents.

After the GenPrint program finishes processing the current transaction, it closes the current output device file. This includes executing any post-batch banner processes. It then continues processing the recipient batch.

If you have assigned a new output device file name using the SetDeviceName function, the system will create and start writing to a new print stream file with that name. The best place to call the BreakBatch function is in the post-transaction banner DAL script.

UniqueString

Use this function to return a 45-character globally unique string.

Syntax UniqueString()

Here is an example:

```
DataPath = GetINIString(, "Data", "DataPath")
Drive = FileDrive(DataPath)
Path = FilePath(DataPath)
UniqueID = UniqueString()
Outputname = FullFileName(Drive, Path, UniqueID, ".PDF")
SetDeviceName(Outputname)
```

USING DAL TO MANIPULATE FILE NAMES

Since you can use DAL functions to read tables and to set device names for output print stream files, this feature further extends DAL functionality by letting you manipulate file names.

For instance, you can get the components of a file name (drive, path, name, and extension) and combine those into a full file name. For example, for computers running Windows file names look like this:



For computers running OS/390, file names look like this:



In this OS/390 example, the drive and extension are omitted, because they are not applicable on OS/390 and the parentheses enclosing `member` are part of the path.

To do this you use these DAL functions:

- FileDrive
- FilePath
- FileName
- FileExt
- FullFileName

All platforms are supported and both the Rules Processor and the Entry system are supported.

Each platform will use platform specific logic to extract or assemble the components. For example, UNIX uses forward slashes and OS390 uses DD names or partitioned dataset names for the *path* and member names for the *name*.

Here are descriptions of these functions:

FileDrive

Use this function to get the drive component of a file name.

Syntax FileDrive("FullFileName")

This function accepts a string containing a fully qualified file name, returns a string that contains the drive component of that file name.

Here is an example:

```
MYDRIVE = FileDrive("d:\mypath\myfile.ext")
```

In this example, MYDRIVE would contain:

"d:"

FilePath

Use this function to get the path component of a file name.

Syntax FilePath("FullFileName")

This function accepts a string containing a fully qualified file name, returns a string that contains the path component of that file name.

Here is an example:

```
MYPATH = FilePath("d:\mypath\myfile.ext")
```

In this example, MYPATH would contain:

"\mypath\"

FileName

Use this function to get the name component of a file name.

Syntax FileName("FullFileName")

This function accepts a string containing a fully qualified file name, returns a string that contains the name component of that file name.

Here is an example:

```
MYNAME = FileName("d:\mypath\myfile.ext")
```

In this example, MYNAME would contain:

"myfile"

FileExt

Use this function to get the extension component of a file name.

Syntax FileExt("FullFileName")

This function accepts a string containing a fully qualified file name, returns a string that contains the extension component of that file name.

Here is an example:

```
MYEXT = FileExt("d:\mypath\myfile.ext")
```

In this example MYEXT would contain:

“.ext”

FullFileName

Use this function to make the full file name.

Syntax FullFileName(“Drive”,“Path”,“Name”,“Ext”)

This function accepts a string containing the drive, path, name, and extension components of a fully qualified file name, assembles them, and returns a string that contains the full file name.

Here is an example:

```
MYFILENAME = FullFileName("d:", "\mypath\","myfile", ".ext")
```

In this example, MYFILENAME would contain:

“d:\mypath\myfile.ext”

NOTE If, in this example, *\mypath* had no trailing slash, the FullFileName function would have added it for you.

Here is an OS/390 example:

```
FullFileName(, "DD:DEFLIB()", "MEMBER")
```

In this example, the result would be:

DD:DEFLIB(MEMBER)

CONTROLLING WIP FIELD ASSIGNMENTS

You can use options in the Trigger2WIP control group to set almost all of the WIP record fields for each transaction.

NOTE Do not try to set the ModifyTime, InUse, or the FormSetID fields of the WIP record. The ModifyTime field is assigned by the system when a WIP record is added or updated. If you need to save a date and time for the transaction, store that information in the CreateTime field, using the hextime X format for the destination as shown in one of the examples.

The InUse field is used internally to prevent multiple people from editing the same transaction. Let the system manage this column.

The FormSetID is assigned by the system when a new WIP transaction is created. Let the system handle this.

The Trigger2WIP control group defines which recipient batch (RCB) transaction fields from the manual batch (those kicked to WIP) are mapped to the corresponding WIP transaction record fields.

The options under the Trigger2WIP control group define the mappings as shown here:

The contents of this field... `< Trigger2WIP >` ...is copied into this field

```
RCBField 1 = WIPField 1
RCBField 2 = WIPField 2
...
```

RCBField represents one of the fields defined by the batch transaction record definition (RCBDFDL.DFD). *WIPField* represents a field defined in the WIP database.

NOTE There may be an external WIP.DFD file that identifies the fields in a WIP record. An external DFD file is not required if you are using the default WIP database layout.

Note, although the normal mapping technique is to name a RCB field on the left side, the left side can name any defined GVM (global variable member). Typically, the only GVMs that exist during GenWIP processing are those defined in the RCB DFD file, but custom applications or single-step WIP systems may have additional GVMs.

The changes in this release support this INI definition and also let you convert data or define a constant value you want to map to a WIP field. For a data conversion, define your INI options as shown here:

```
< Trigger2WIP >
RCBField = WIPField; input format ; output format;
```

The conversion information must appear on the right side of the INI option, after the WIPField name definition. Separate it from the named variable with a semicolon. Here is an example:

```
RunDate = CREATETIME;DD4;X
```

The first semicolon denotes the input format of the data. The second separates the desired output format. In this example, the input format **DD4** means the source data is a date field in the format **DD4**, which is YYYYMMDD.

The output format `X` indicates you want to convert the date value to the internal HEXTIME format used in the WIP CreateTime field.

NOTE For more information selecting from the pre-defined date formats or defining your own, see the [Rules Reference](#)

Although conversions are often used to change date formats, you can also use them to do additional formatting. The system supports a simple C style `sprintf` (`%s`) and constant text formatting, like `%s`, `%10.10s`, `%-38.38s`, and so on. The system does not support any of the other C style formats flags that assume non-text data or asterisk (variable width) designations.

Here is an example:

```
EFFVALUE = APPData; ;(%s%%)
```

Suppose in this example, that EFFVALUE contains the text `10%`. The resulting value mapped into the APPData field will read `10%`.

NOTE You must use two percent signs (`%%`) to represent a single percent sign in the output. The system only supports a string `%s` type format. It does not support numeric data formats of any type.

Normally, the left side of the INI option names a field from the RCB file definition. You can also enter `NULL` as a keyword to mean there is no corresponding RCB data field to associate with the WIP field. This lets you assign the constant data to the WIP field, as shown here:

```
NULL = DESC; ;ABC123 HERE WE GO
```

This example shows how to assign the constant `ABC123 HERE WE GO` to the DESC field of the WIP record. `NULL` indicates there is no source variable to associate with this destination field.

You can also use INI built-in functions to provide a constant value to map to the field. For example:

```
NULL = CURRUSER; ;~GETENV USERNAME
```

INI built-in functions are preceded with a tilde (`~`). This example executes the `GETENV` built-in INI function, which gets the environment variable `USERNAME`. If you assume the variable contained the text `TOM` the WIP variable `CURRUSER` would be assigned `TOM` after execution of the built-in function.

These options show the defaults used if the Trigger2WIP control group does not override the variables:

```
< AFG2WIP >
  StatusCode = WIP
  RecordType = NEW
  UserID     = DOCUCORP
```